LitasCloud

Cloud Gaming Service

Technical Specification

Team members

Karolis Paškevičius
Eigintas Urbanavičius
Kęstutis Karneckas
Mantas Delgiado
Dovydas Valantiejus

Vilnius University
November 7, 2026

Contents

1 Introduction **3**

1.1 Overview . . . 3
1.2 Problem and Motivation . . . 3
1.3 How the system works . . . 3
1.4 Target Audience . . . 3
1.5 Goals and scope . . . 3

2 Business Requirements **4**

2.1 Business goals and success measures . . . 4
2.2 Stakeholders Analysis . . . 5
2.3 Current activity analysis . . . 5
2.4 User needs . . . 7
2.5 MoSCoW analysis . . . 7
2.5.1 Must have . . . 7
2.5.2 Should have . . . 10
2.5.3 Could have . . . 11
2.5.4 Won't have . . . 12

3 Business Flows **14**

3.1 Host Starts Offering . . . 14
3.2 Player Requests a Session . . . 15
3.3 Connection Setup and Recording . . . 15
3.4 Ending a Session and Connection Loss . . . 19
3.5 Credits Settlement . . . 19

4 Non-Functional Requirements **22**

5 High-Level Architecture **23**

5.1 System Overview . . . 23
5.2 Packages and Modules . . . 25
5.3 Class Diagrams . . . 28
5.4 State Machines . . . 32

6 API Contract **34**

7 Data Model **34**

7.1 Enumerations . . . 35
7.2 Entities . . . 36
7.3 Relationships . . . 41
7.4 Integrity rules . . . 41
7.5 Derived values and data outside the database . . . 42

7.6 Local data of the Host Agent 43

8 UI/UX Wireframes **43**

8.1 Player Wireframes 43

8.1.1 A1 — Sign in 45

8.1.2 A2 — Create account 46

8.1.3 P1 — Browse available games 46

8.1.4 P2 — Waiting for Host approval 47

8.1.5 P3 — Connecting to the game 48

8.1.6 P4 and P4a — Play the streamed game 49

8.1.7 P5 — Recovering a dropped connection 50

8.1.8 P6 — Result screens 50

8.2 Operator Wireframes 53

8.2.1 O1 — Review and lift Host blocks 53

8.2.2 O2 — Review session recordings 53

8.3 Host Agent Wireframes 54

8.3.1 H1 — Set up an offering 55

8.3.2 H2 — Manage allowed games 56

8.3.3 H3 — PC readiness check 56

8.3.4 H3a — PC check failed 56

8.3.5 H4 — Offering is available 57

8.3.6 H5 — Approve a Player request 57

8.3.7 H6 — Active hosting session 57

8.3.8 H7 — Confirm an early disconnect 58

8.3.9 H8 — Hosting is temporarily blocked 58

8.3.10 H9 — Session result 59

1 Introduction

1.1 Overview

LitasCloud is a cloud gaming service that lets a person play a game running on another person's PC through a web browser. The host PC runs the game and streams its video to the player. The player's keyboard and mouse input travels back to the host in real time, so the player needs no game installation and no powerful hardware.

The service has two roles:

- **Host:** the owner of a PC who shares it and approves incoming connections.
- **Player:** a user who connects to a shared PC from a browser and plays.

1.2 Problem and Motivation

Modern video games demand powerful hardware, and not everyone has it. Many people own a PC that is too weak to run the games they want to play, and upgrading it costs more than they are willing to spend. Meanwhile, powerful PCs often sit idle for most of the day. The hardware exists, but is not available for everyone. Commercial cloud gaming services address this with data centers, but they are not an option for everyone due to their price or region. LitasCloud takes a more personal approach. Instead of renting a remote server, a person lends their own PC to a player. The system includes safeguards, so the player and host do not risk their personal information or hardware. Player can play a game directly in their browser without downloading anything.

1.3 How the system works

The Host runs a small program on their Windows PC. The Host chooses the time amount to lend their PC for. Then host agent connects to a server, transfers information such as (PC lend start and end dates, PC specifications and what games available this host has) and waits for a player to connect to it. When a player presses play a central server connect both player and a host after authenticating and authorizing both of them. After that host PC start a selected game in the hidden mode for host, so the host does not see what connected player plays, starts the stream and waits for player input while reacting to it. It is forbidden for a host to end a session before the lend date ends and should be punished by a temporary block.

1.4 Target Audience

- **Players** with low-end computers who want to play games they could not otherwise run.
- **Hosts** with capable PCs willing to share their machine with players for a reward.

1.5 Goals and scope

The first version is a working prototype. Its main goal is to let a Player run a simple game, such as Snake, in the browser while it runs on a remote PC, with latency low enough for the game to be playable. It is designed for current desktop versions of Chrome, Firefox, and Edge, and for Hosts on Windows 10 or 11. One Host Agent serves one Player at a time.

2 Business Requirements

This section states what LitasCloud must do for the people and parties who use or run it. It builds on the Introduction (Section 1): a Host lends a Windows PC for a chosen time, a Player plays a game on it through a browser, and the Host is rewarded. Requirements are written as user stories and prioritized with MoSCoW: Must have (the system does not work without it), Should have (important, included if time allows), Could have (desirable, not needed for the first version) and Won't have (explicitly out of scope).

2.1 Business goals and success measures

Business goal. Let people whose computers are too weak for modern games play them in a browser, using the idle PCs of other people, while the Host stays in control of their machine and is rewarded for lending it. Powerfull server are not needed because the server only authenticates player and a host, receives information and connects them. Video, audio, and input travel directly between the two peers.

The first version is a prototype. Success is measured with the targets below.

Table 1: Overview of System Goals, Descriptions, and Target Measures

ID	Goal	Measure and Target
G1	Playable game	A Player plays "Snake" game for 10 minutes without disconnecting at 720p, with FPS and latency meeting NFR-01 and NFR-02.
G2	Fast connection	The first video frame appears within 10 seconds after the Host approves.
G3	Easy for Players	At least 4 of 5 first-time Players, without help, create an account, choose a game, waits for approval and reach the stream within 5 minutes.
G4	Easy for Hosts	At least 4 of 5 first-time Hosts log in, choose a game and a duration and start offering their PC within 3 minutes of opening the Host Agent.
G5	Host stays in control	Disconnect stops stream and input within 2 seconds (NFR-06);
G6	Fair rewards	In every completed session, the credits the Host earns the credits the Player spends - 5% (S4).

2.2 Stakeholders Analysis

Table 2: Stakeholders, Categories, and Needs

Category	**Stakeholder / Entity**	**Needs & Goals**
Primary users	Players: people with low-end computers	Play demanding games without installing anything or buying hardware; start quickly; see that the connection is good; pay a fair price.
Primary users	Hosts: owners of capable PCs	Decide when and for how long to lend the PC; approve who connects; be protected from misuse of the PC; receive a reward.
Secondary	Operators: the team that runs the Signaling Server	Low running cost (no video relay except as a fallback); ability to stop abuse and unreliable Hosts.
Indirect	Host's household and network	The session must not hand control of the PC to the Player or overload the home connection unnoticed.
Indirect	Game publishers	Games stay licensed to the Host; the service does not copy or distribute games.
Competitors	Data-center cloud gaming (e.g., GeForce NOW, Xbox Cloud Gaming); remote-play tools (e.g., Steam Remote Play, Parsec)	They already stream games. LitasCloud differs by running in a plain browser, using people's own idle PCs and rewarding the lender.

2.3 Current activity analysis

Player scenario. Jonas, 20, a student with an 8 GB RAM laptop, wants to play "Minecraft" with demanding graphics settings. The laptop stutters at 15 FPS. A subscription cloud service needs a payment method and an installed client. He searches the internet and finds a subscription service "Geforce Now", however, it needs to be installed and it costs at least 10.99$ a month to use. Because Jonas would like to play Minecraft in his free time that is usually weekends and cannot spend that much money for a monthly subscribtion. Thus, he stays with playing "Minecraft" with his own laptop while the game is stuttering.

Problems and improvement opportunities:

- Hardware he cannot afford to upgrade is not able to run the games he wants to play.
- Existing options need installation, a subscription or technical setup.
- LitasCloud should need only a browser,a registration, and a Host user.

Host scenario. Tadas, 24, owns a gaming PC that is idle while he is at work. He would lend it, but he worries that a stranger could open programs, see his files or that the PC will be busy when he needs it. He researches on th internet how could he earn money, however, the only options he finds are crypto mining or sharing connection for money. These crypto applications are suspicios, often gets flagged by "Windows Defender" and has questionable reputation, thus Tadas is scared of his files being stolen, his PC and network being used for crimes. He has no way to limit the time, restrict what can be run.

Problems and improvement opportunities:

- Idle hardware earns nothing.
- Full remote desktop tools give the guest control of the whole PC.
- A host who disappears halfway through a session spoils the Player's time, so the service needs a punishment that discourages it.

Player scenario. Mindaugas, 19, wants to play a fast-paced action game on his budget laptop. Before connecting, he needs to know whether the available Host PC actually has a strong GPU and if the session will lag. Once the stream starts, he needs to see his live ping and FPS overlay so he knows his inputs are responsive. Because LitasCloud routes video and input directly between Mindaugas and the Host via peer-to-peer technology, the operator avoids expensive server relay costs and Mindaugas gets crisp, low-latency gameplay.

Problems and improvement opportunities:

- Players cannot verify if a Host PC is powerful enough before paying or connecting.
- High latency or hidden lag spikes ruin fast-paced gameplay without clear connection metrics.
- P2P video and input streaming keeps server infrastructure costs low while maintaining sub-second input responsiveness.

Host scenario. Dovydas, 26, works from home on his dual-monitor desktop setup. He wants to earn extra income by renting out his idle GPU power during work hours, but he cannot have a remote player's game taking over his main monitor or receiving mouse inputs while he works. He is also worried about unreliable players trying to abuse his system.

Problems and improvement opportunities:

- Hosts cannot lend their PC if the remote session hijacks their primary screen or controls.
- Operators must automatically detect and filter out abusive players or unreliable host nodes to maintain platform trust.

2.4 User needs

Below in the table there are provided user needs form user stories where the need is linked to the source story.

Table 3: User Needs and Sources

ID	Need	Source Story & Context
UN1	A Player wants to play a game their PC cannot run, without installing anything.	Jonas (Player scenario 1)
UN2	A Player wants to start quickly: log in, choose a game, wait for approval, connect and play.	Jonas (Player scenario 1)
UN3	A Player wants responsive controls and to see how good the connection is.	Mindaugas (Player scenario 2)
UN4	A Player wants to play from home, not only on the Host's local network.	Jonas & Mindaugas scenarios
UN5	A Player wants to know before connecting which game is offered and whether the Host PC is capable.	Mindaugas (Player scenario 2)
UN6	A Player wants a clear, fair cost for the time played.	Jonas (Player scenario 1)
UN7	A Player wants the session to last for the whole time they booked.	Tadas (Host scenario 1, problem 3)
UN8	A Host wants to control who uses the PC.	Tadas (Host scenario 1)
UN9	A Host wants the PC protected: limited input, only allowed games, game closed at the end.	Tadas (Host scenario 1)
UN10	A Host wants to choose when and for how long the PC is lent.	Tadas (Host scenario 1)
UN11	A Host wants a reward for lending.	Tadas (Host scenario 1)
UN12	Host and Player both want the Player's game kept off the Host's screen, so neither disturbs the other.	Dovydas (Host scenario 2)
UN13	A Host wants to be able to stop the session immediately in an emergency.	Tadas & Dovydas scenarios
UN14	The operator wants to prevent abuse and unreliable Hosts.	Dovydas (Host scenario 2) & Operator
UN15	The operator wants low running cost: video and input go peer to peer.	Mindaugas (Player scenario 2) & Operator

2.5 MoSCoW analysis

The MoSCoW analysis prioritizes system features and requirements into four distinct categories. This framework establishes clear delivery expectations for the project scope, helping developers, stakeholders, and operators understand what is mandatory for a functional release and what is deferred for future iterations.

2.5.1 Must have

Requirements in this section represent the non-negotiable core functionality of the platform. Without these features the system cannot function or be considered operational.

Table 4: Must have requirements

ID	User story	Acceptance criteria	Traces to
M1	As a Host, I want to offer my PC with a chosen games list and a duration, so that Players can find and request it.	A logged-in Host who has chosen a duration (M9) and a games list (M10) presses "Start hosting". The Host Agent registers with the server (games list, duration). The Host games then appears in the games list as Available. A Host has at most one active offering or session. The Host can cancel the offering while no session is running. If the Host Agent loses its server connection, the offering disappears within 10 s.	UN5, UN10, G4
M2	As a Player, I want to browse available gsames and press play, so that I can start quickly.	A logged-in Player sees only available games with remaining session length. Pressing play sends a join request. The server checks that both sides are authenticated and that the Host is still available, then reserves the host so no second Player can take it. A Player has at most one pending or active request. The player sees a waiting screen.	UN2, UN5, G3
M3	As a Host, I want to approve or reject a connection request, so that nobody uses my PC without permission.	The request shows the Player's username and the game. No video is sent and no input is accepted until approval. A rejection or no answer within 30 s shows a message to the Player. The Host then returns to Available without having to start a new session.	UN8
M4	As a Player, I want to see the game in my browser, so that I can play without installing anything.	After approval the Host agent starts the chosen game, captures its picture, and streams it P2P at 720p and at least 30 FPS on to a players session. The first frame appears within 10 s of approval. Playback needs no plugin.	UN1, G1, G2
M5	As a Player, I want my keyboard and mouse to control the game on the Host's PC, so that I can play.	User can use letters, digits, arrow keys, Space, Enter, Escape, Shift. Also they can use mouse movement, left and right clicks, and scroll. Everything else, and any combination that poses a treath to Hosts pc with Ctrl, Alt or the Windows key, is ignored. Input is applied only while the game window is the target.	UN1, UN3, UN9, G1
M6	As a Host, I want to end the session at any time, so that I stay in control of my PC. Knowing that I could be punished for this action	The Disconnect button ends the session within 2 s. Player input stops immediately and the browser tells the Player. On any session end the Host Agent releases all held keys, stops capture, and closes the game. If either side loses the connection (no heartbeat for 5 s), the other side is informed and the session enters Reconnecting (S6), it end if the connection is not back within 20s. The Host can start a new offering afterwards. The Player has a Leave button.	UN13, G5

Table 4: Must have requirements

ID	User story	Acceptance criteria	Traces to
M7	As a Player, I want to see latency and FPS on screen, so that I know the connection quality.	Round-trip latency (ms) and FPS are shown in a corner of the video and updated every second.	UN3, G1
M8	As a Host or Player, I want to register and log in, so that the service knows who hosts and who plays.	An account has a unique username and a password, stored as a salted hash. A login is required to offer or join. Host actions are allowed only to a logged-in Host and Player actions only to a logged-in Player. A wrong login shows an error.	UN2, UN8, UN14, G3
M9	As a Host, I want to choose for how long I lend my PC, so that I decide when it is mine again.	Before offering, the Host picks a duration (15 min to 4 h). Both sides see the remaining time. The Player is warned 1 min before the end. At the end the session closes and the game is closed. Time counts from the first video frame.	UN7, UN10, G4
M10	As a Host, I want to offer only games I choose, so that only those can run on my PC.	The Host adds games (name and program path) to the Host Agent. The prototype supports multiple simple games. The agent launches only listed games. The Player cannot start any other program.	UN9, G1, G4
M11	As a Host or Player, I want the Player's game to run hidden from my screen, so that neither of us disturbs the other.	During a session the Host's desktop and other windows are never captured, and the stream contains only the game. The Host can keep working normally. The Host's own keyboard and mouse do not reach the game, and the Player's input does not reach the Host's other applications.	UN9, UN12
M12	As a Player, I want the server to connect me directly to the Host from my own network, so that I can play from home with low latency.	The server relays only SDP and ICE messages, and only between the two participants of one session. Video and input travel P2P and are encrypted (DTLS/SRTP). There are no pairing codes or connection tokens. Access is decided by authenticated accounts and session state.	UN1, UN4, UN15, G2
M13	As the operator, I want a copy of every session stream saved, so that abuse can be investigated.	A server-side Recording Agent joins each active session as an extra peer and receives a copy of the Host's video track. The recording is saved with sessionId, Host, Player, and start and end time. The Recording Agent uses its own connection, so frame drops or failure there never degrade the Player's stream. If recording fails, the session continues and is flagged. Recordings are video only, readable only by operators, and auto-deleted after 7 days unless a report is open. Hosts and Players are told about recording before connecting.	UN14, UN15

2.5.2 Should have

High-priority requirements that significantly improve system usability, reliability, and coverage. While essential for a complete user experience their temporary absence does not break the core functionality of the system.

Table 5: Should have requirements

ID	User story	Acceptance criteria	Traces to
S1	As a Player, I want to cancel my join request while waiting, so that I am not stuck if I changed my mind or the Host is slow.	The waiting screen has a Cancel button. Pressing it removes the request and releases the Host reservation within 2 s. The Host's approval dialog (M3) closes, and the Host returns to Available. The Player can then send a new request to any Host.	UN2, UN8
S2	As a Player, I want the connection to work even when my network blocks direct connections, so that I can play from home.	The browser and Host Agent first try a direct path with a public STUN server. If it fails, they use a TURN relay (coturn). The Player sees "Connecting via relay" when relay is used. Video and input stay encrypted (DTLS/SRTP).	UN4, UN15, G2
S3	As an Operator, I want Hosts who end sessions early to be blocked for a while, so that Players can rely on the full time they booked.	If a Host presses Disconnect before the booked time ends (M9), a confirmation dialog first warns about the block. After confirming, the session ends within 2 s as usual (M6, G5). The Host cannot start a new offering (M1) for 1 hour. A repeat within 30 days blocks for 24 hours. The Host sees the reason and the remaining block time. An Operator can lift a block after review. A Player leaving or a lost connection never blocks the Host.	UN7, UN14, G5
S4	As a Host or Player, I want the service to count credits for played time, so that cost and reward are clear and fair.	A Player needs enough credits for the chosen duration before the join request is sent (M2). Credits are charged per full minute played, counted from the first video frame (M9). The Host earns the credits spent minus a 5% service fee. If the Host ends the session early, the Player pays only for the time played. Balances never go below zero. Both sides see their balance and the price per minute before connecting.	UN6, UN11, G6
S5	As a Host, I want to test my PC before offering it, so that I do not offer a PC that cannot run the stream.	When the Host presses "Start hosting" (M1), the Host Agent checks screen capture, upload speed (at least 4 Mbps for the 720p stream), and server connection. If a check fails, the Host sees what failed and what to do next (NFR-15), and the offering is not created. The results are kept for the Host listing (C5).	UN5, UN9, G2, G4

Table 5: Should have requirements

ID	User story	Acceptance criteria	Traces to
S6	As a Player, I want a short connection drop to be recovered, so that I do not lose my session because of a brief network problem.	If the heartbeat stops for 5 s (NFR-07), the Player sees "Reconnecting…" for up to 20 s and the Host Agent holds the session without accepting input. If the connection returns, the stream resumes and the Host Agent keeps all keys released. If not, the session ends as in M6. The session time keeps counting during the drop.	UN3, UN7

2.5.3 Could have

Desirable enhancements that provide added value, quality-of-life improvements, or enhanced host control. These features are implemented if time, resources, and technical feasibility allow, but can easily be deferred without impacting primary workflows.

Table 6: Could have requirements

ID	User story	Acceptance criteria	Traces to
C1	As a Player, I want to hear the game's sound, so that the game feels complete.	The Host Agent captures audio only from the running game, never the Host's microphone or other applications. It sends the audio as a WebRTC audio track (Opus) over the same P2P connection as the video. The Player has a mute button and a volume slider, and the browser starts muted until the Player's first click (browser autoplay rules). Audio stops within 2 s of session end (NFR-06). Audio is not saved by the Recording Agent (M13 is video only).	UN1, UN3, G1
C2	As a Player, I want to use a gamepad, so that I can play games designed for controllers.	The browser reads a connected gamepad with the Gamepad API and sends button and axis states over the control data channel. The Host Agent applies them through a virtual gamepad, only while the game window is the target (M5, M11). It releases all buttons and centers all axes on session end or connection loss (M6). The Host can turn gamepad support off per game (M10).	UN1, UN3, UN9
C3	As a Player, I want to choose the video quality, so that I can lower it on a slow connection or raise it on a fast one.	The Player picks a preset (e.g. 480p/30, 720p/30, 720p/60) before or during a session. The Host Agent applies it within 3 s without ending the session. The default stays 720p/30 (M4). A preset above the maximum the Host's PC passed in the self-test (S5) is not offered. The latency/FPS overlay (M7) keeps working, and frame dropping (NFR-04) applies at every preset.	UN3, G1

Table 6: Could have requirements

ID	User story	Acceptance criteria	Traces to
C4	As a Player, I want a fullscreen button, so that the game fills my screen.	A button in the stream view toggles browser fullscreen. The Leave button and the latency/FPS overlay stay reachable in fullscreen. Key and mouse capture (M5) keep working. Escape exits fullscreen only in the browser and is not sent to the game while exiting.	UN1, UN3
C5	As a Player, I want to see the Host PC's specifications before I press play, so that I know the PC can run the game.	Each available Host in the list (M2) shows CPU, GPU, RAM and recent connection quality (average latency and the upload speed measured in S5). The Host Agent reads the specs and sends them when registering (M1). The Player sees them before sending a join request. The Host cannot edit the numbers by hand.	UN5, UN3
C6	As a Player, I want to ask for more time near the end of a session, so that I can keep playing without a new booking.	In the 1-minute warning (M9) the Player can send an extension request. The Host can accept or decline within 30 s (like M3). If accepted, the remaining time grows by the requested amount, the total stays within the 4 h maximum (M9), and the Player must have enough credits (S4). If declined or ignored, the session ends at the original time.	UN7, UN10, UN6
C7	As a Host or Player, I want to rate the other side after a session and report a problem, so that unreliable or abusive users can be found.	After a session ends, each side can give a 1-5 rating and an optional report. Ratings are shown on Host listings (M2) and in the Host's approval request (M3). A report links to the session recording (M13) and keeps it past the 7-day auto-delete while the report is open. Operators can review reports and suspend accounts. A Host block from S3 is separate and automatic.	UN14, UN8
C8	As a Host, I want to set a weekly availability schedule, so that my PC starts offering itself only at the times I choose.	The Host sets days, times and a duration (15 min to 4 h, M9) once. The Host Agent starts the offering automatically at those times if the Host is logged in, not blocked (S3) and no session is running. The Host can cancel any scheduled offering (M1). The limit of one active offering per Host still applies.	UN10, G4
C9	As a Host or Player, I want to see my session history, so that I can check time, credits and who I played with.	A logged-in user sees a list with date, duration, game, the other user's username and the credits earned or spent (S4). Users see only their own sessions. Recordings (M13) are not shown.	UN6, UN11

2.5.4 Won't have

Explicitly excluded features and technical boundaries for the current system version or prototype. Clearly defining what will not be built manages stakeholder expectations, prevents scope creep, and keeps development focused on achievable targets.

Table 7: Won't have requirements

ID	Excluded item	Reason
W1	Several Players in one session (co-op, spectators)	One Host Agent serves one Player at a time (M2). Multiple streams would need more upload bandwidth and a different input model.
W2	Games protected by anti-cheat systems	Such games block or flag input injection and screen capture, and could get the Host's game account banned.
W3	Payments with real money, withdrawals, or exchanging credits for money	Credits (S4) only measure time in this version. Real payments need legal, tax and payment-provider work that is outside the prototype.
W4	Mobile browsers and touch controls	The client targets desktop Chrome, Firefox and Edge only (NFR-16).
W5	Safari and other browsers not listed in NFR-15	Not tested in this version.
W6	Host PCs on macOS or Linux	The Host Agent supports Windows 10 and 11 only (NFR-17).

3 Business Flows

This section shows how the requirements of Section 2 work together over time. Each flow is a sequence diagram. Solid arrows are requests or actions, dashed arrows are responses or notifications, and thick blue arrows are peer-to-peer WebRTC traffic. Rectangles labelled *alt*, *opt* and *loop* mark alternatives, optional steps and repetition. Yellow notes describe work done inside a single participant.

Participants. *Host* and *Player* are the system users. The *Host Agent* is the program on the Host's Windows PC. The *Browser* is the Player's web client. The *Signaling Server* authenticates accounts, keeps offerings, requests and sessions, relays SDP and ICE messages, and settles credits. The *Database* stores accounts, offerings, sessions, credits and penalties (Section 7). *STUN/TURN* (coturn) helps the two peers find each other. The *Recording Agent* is the operator's server-side peer that saves a copy of the video (M13).

Life of one session. Five flows cover a complete use of the service:

1. The Host starts an offering (Section 3.1).
2. A Player requests it and the Host decides (Section 3.2).
3. The two peers connect and the video starts (Section 3.3).
4. The session ends, normally or because of a failure (Section 3.4).
5. Credits are settled (Section 3.5).

Timers and limits used in the flows. The values come from the requirements in Section 2. The only exception is the 3-second heartbeat interval, a design choice that satisfies the 10-second rule of M1.

Item	Value	Source
Offered duration	15 min – 4 h	M9
Offering heartbeat (Host Agent → server)	every 3 s; offering ends after 10 s without one	M1
Host decision on a request	30 s	M3
First video frame after approval	within 10 s	M4, G2
Ping/pong heartbeat during a session	every 1 s; lost after 5 s	M6, NFR-07
Reconnection window	up to 20 s	S6
Warning before the end	60 s	M9
Disconnect takes effect	within 2 s	M6, NFR-06
Host block after an early Disconnect	1 h; 24 h if repeated within 30 days	S3
Platform fee	5 %	S4
Recording retention	7 days (longer while a report is open)	M13, C7

3.1 Host Starts Offering

The Host logs in, chooses the games and the duration, and presses "Start hosting". The Host Agent first runs the self-test (S5), then registers the offering. From then on it sends a heartbeat every 3 seconds. The offering appears in the Games list as `AVAILABLE` until a Player reserves it, the Host cancels it, or the heartbeat stops (Figure 2).

- A wrong username or password shows an error and the flow ends (M8). Host actions are allowed only to a logged-in Host.

- The Host Agent offers only games from its own list and only a duration between 15 minutes and 4 hours (M9, M10). Before registering, it checks screen capture, upload speed (at least 4 Mbps) and the server connection, and reads the PC specifications (S5, C5). If a check fails, the Host sees what failed and what to do, and no offering is created.
- The server accepts the registration only for an account with the Host role that has no active block (S3) and no other active offering or session (M1). Otherwise it answers with an error and nothing is stored.
- The Host can cancel an `AVAILABLE` offering at any time (M1). Once a Player has reserved it, the Host answers the request instead (Section 3.2).
- If the server receives no heartbeat for 10 seconds, the offering becomes `ENDED` and disappears from the list (M1). If it was `RESERVED`, the waiting request is rejected and the Player's credit hold is released.

3.2 Player Requests a Session

The Player sees only `AVAILABLE` offerings, with the games, the PC specifications, the duration and the price per minute (M2, C5). After choosing a game and pressing Play, the server checks the request and, if it is valid, reserves the Host and holds the Player's credits in one atomic step. The Host Agent then asks the Host for approval (Figure 1).

- The request is refused with an error code if the Player is not logged in or the account is disabled, already has a pending or active request, the offering is no longer `AVAILABLE`, or the credits do not cover the price per minute for the offered duration (M2, S4). Because the reservation is atomic, two Players can never reserve the same Host.
- The Host sees the Player's username and the chosen game. No video is sent and no input is accepted before approval (M3).
- If the Host approves, the request becomes `APPROVED`, the offering becomes `IN_SESSION` and a session is created in state `CONNECTING`. Connection setup follows (Section 3.3).
- If the Host rejects, or does not answer within 30 seconds, the request becomes `REJECTED` or `TIMED_OUT`. The hold is released, the offering returns to `AVAILABLE` without the Host starting anything new, and the Player sees a message (M3). On a timeout the server also closes the Host's dialog.
- While waiting, the Player can press Cancel (S1). The request becomes `CANCELLED`, the hold is released and the Host's dialog closes. The Player can then send a new request to any Host.

3.3 Connection Setup and Recording

After approval the server sends the Host Agent and the Browser the STUN and TURN details. The Host Agent launches the chosen game on a hidden desktop and starts capturing only the game window (M11). The two peers then exchange SDP and ICE messages through the server. The server only relays these messages, and only between the two participants of one session (M12). Video and input never pass through the server (Figure 3).

- The peers try a direct path first. If it fails (for example, with a symmetric NAT), both use a TURN relay and the Player sees "Connecting via relay" (S2). Video and input stay encrypted with DTLS/SRTP in both cases.
- When the first video frame is shown, the Browser reports it. The session becomes `ACTIVE` and the booked time starts counting (M9).
- If no frame arrives within 10 seconds of approval, the session ends with reason `setup_failed` (M4, G2). The Player is not charged (Section 3.5).
- Once the session is `ACTIVE`, the Recording Agent joins as an extra peer and receives a copy of the Host's video track over its own connection (video only, M13). If recording fails, the Player's stream is not affected: the session continues and is flagged.

Flow 2: Player requests a session (M2, M3, S1, S4)

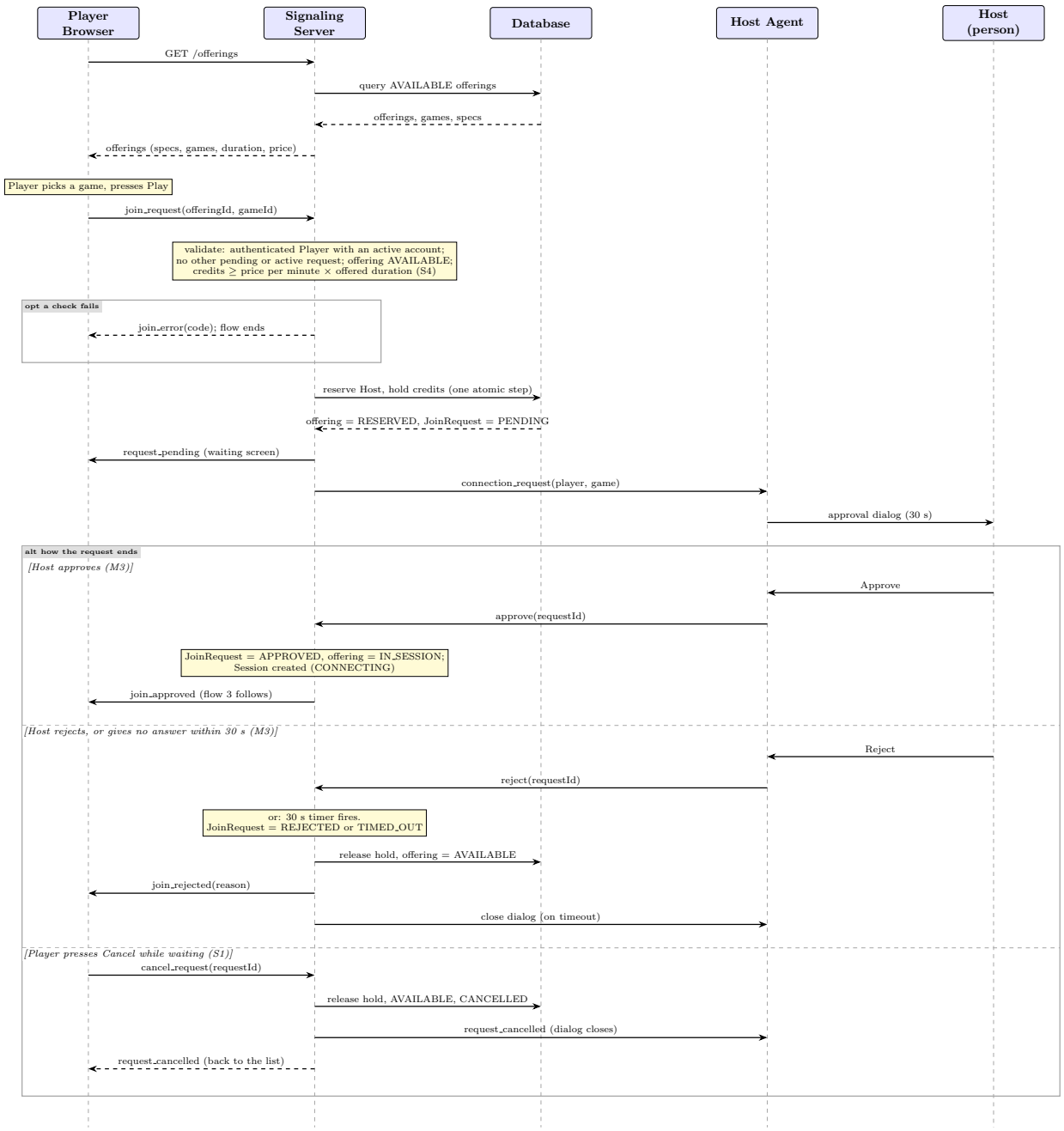


Figure 1: Player requests a session

Flow 1: Host starts offering (M1, M8, M9, M10, S5)

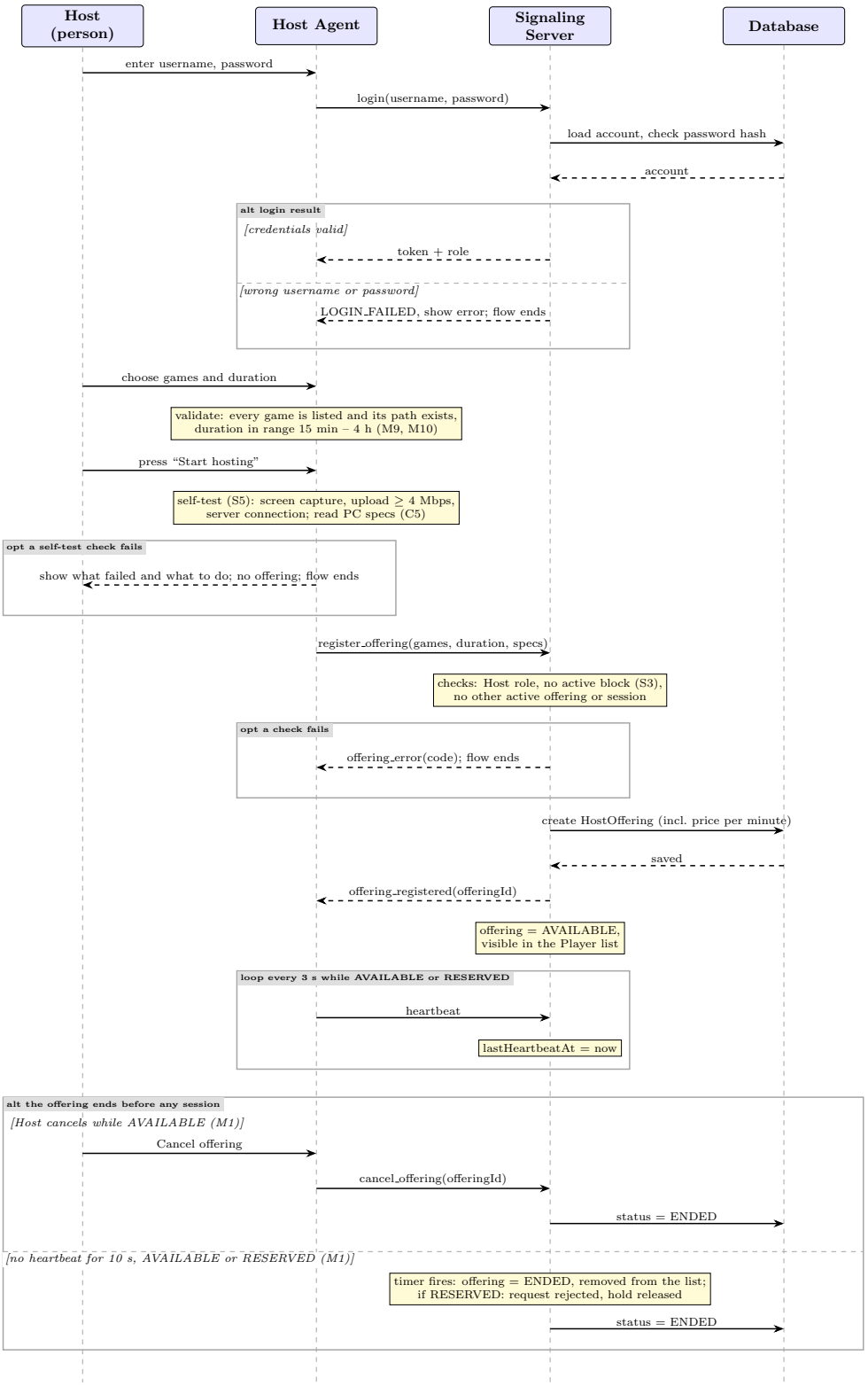


Figure 2: Host starts offering

Flow 3: Connection setup and Recording Agent (M4, M11, M12, M13, S2)

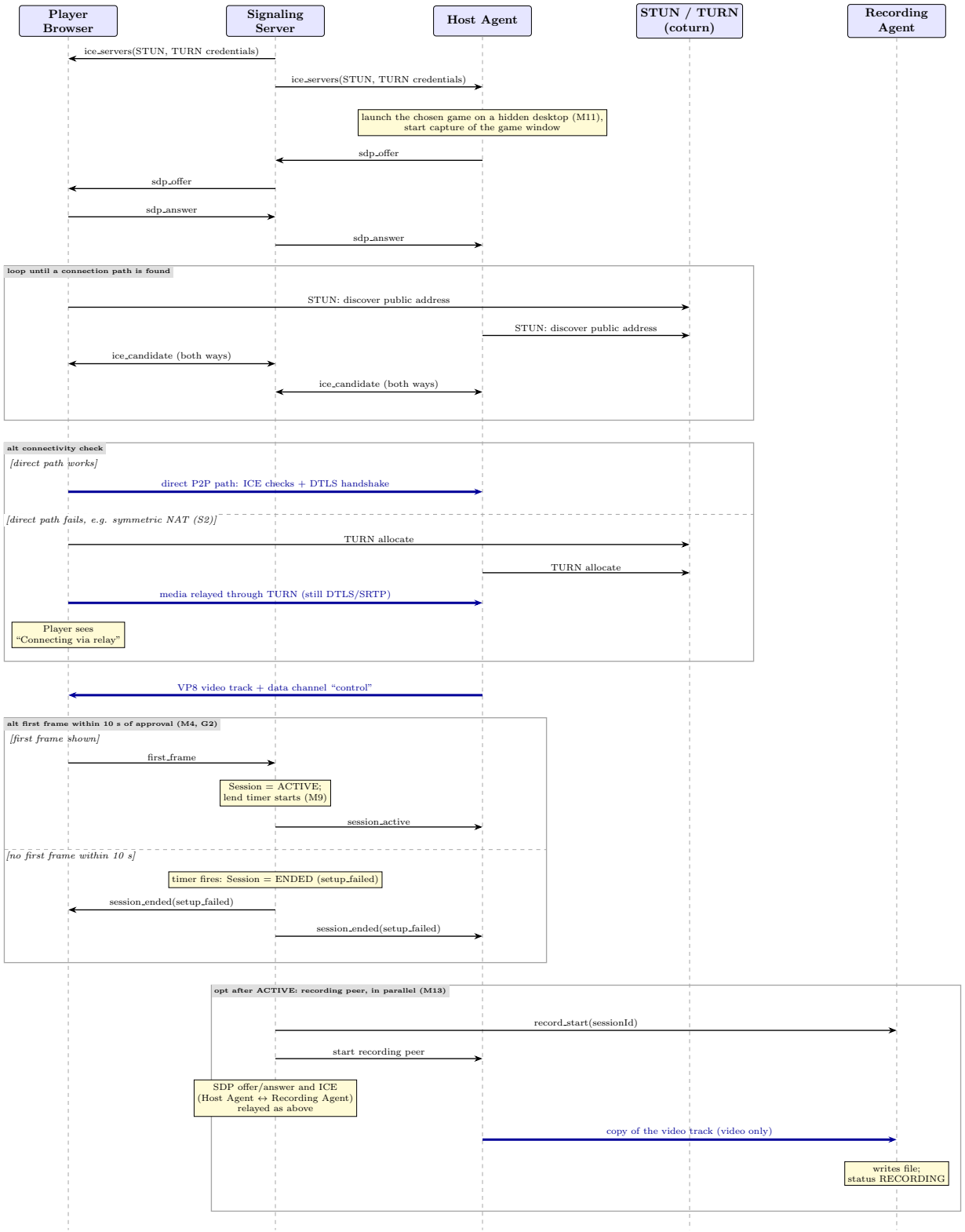


Figure 3: Connection setup and Recording Agent

3.4 Ending a Session and Connection Loss

A session ends for one of the reasons in Table 9. Whatever the reason, the same end procedure runs: the server stores the end reason and the end time, informs both sides, tells the Recording Agent to finish the file, and starts credit settlement (Figure 4).

- **Host Disconnect.** If the booked time has not ended, a confirmation dialog first warns about the block (S3). After confirmation the Host Agent stops the stream and input at once, within 2 seconds (M6, NFR-06), without waiting for the server. The server then stores a penalty. A Player leaving or a lost connection never blocks the Host.
- **Player leaves.** The Leave button, or closing the page, ends the session. If the page is closed without a message, the missing heartbeat ends it.
- **Connection loss.** The Browser sends a ping every second and the Host Agent answers with a pong. After 5 seconds without one, the Player sees "Reconnecting…", and the Host Agent blocks input and releases all held keys. If the heartbeat does not return within 20 seconds, the session ends with reason `connection_lost` (S6). The booked time keeps counting during the drop.
- **Booked time elapsed.** Both sides see the remaining time. One minute before the end, both are warned (M9). At the end the session closes with no penalty.
- **End procedure.** The Host Agent stops input, releases all held keys, stops capture and closes the game (M6). The Browser shows an ended screen with the reason. The recording is finalized and gets a deletion date 7 days later (M13). Afterwards the Host can start a new offering (unless blocked, S3).

Table 9: End reasons

endReason	**Trigger**	**Host blocked**	**Player pays**
host_disconnected	Host presses Disconnect	yes, if time remained	time played
player_left	Leave button or page closed	no	time played
connection_lost	no heartbeat for 5 s, not back within 20 s	no	time played
duration_elapsed	booked time over	no	full duration
setup_failed	no first frame within 10 s	no	nothing

3.5 Credits Settlement

Credits only measure time in this version (W3). When a Player sends a request, the server holds the price of the offered duration (price per minute × minutes). When the session ends, the Session Manager asks the Credit Service to settle it (Figure 5).

- **Billable time.** Credits are counted from the first video frame: billable minutes are the time between the first frame and the end, rounded up to whole minutes. The cost is billable minutes × price per minute, and never more than the held amount, so balances never go below zero (S4).
- **Split.** The platform fee is 5 % of the cost, rounded down. The Host receives the cost minus the fee, so the Host payout plus the fee always equals the cost (G6).
- **One transaction.** Releasing the hold, debiting the Player, paying the Host, booking the fee and marking the session as settled happen in one database transaction. A session is therefore never settled twice.
- **No charge.** If the session never became `ACTIVE` (the request was rejected, timed out or cancelled, or setup failed), the hold is only released.
- **Early end.** If the Host ends the session early, the Player pays only for the time played.

Flow 4: Session end and failure paths (M6, M9, M13, S3, S6)

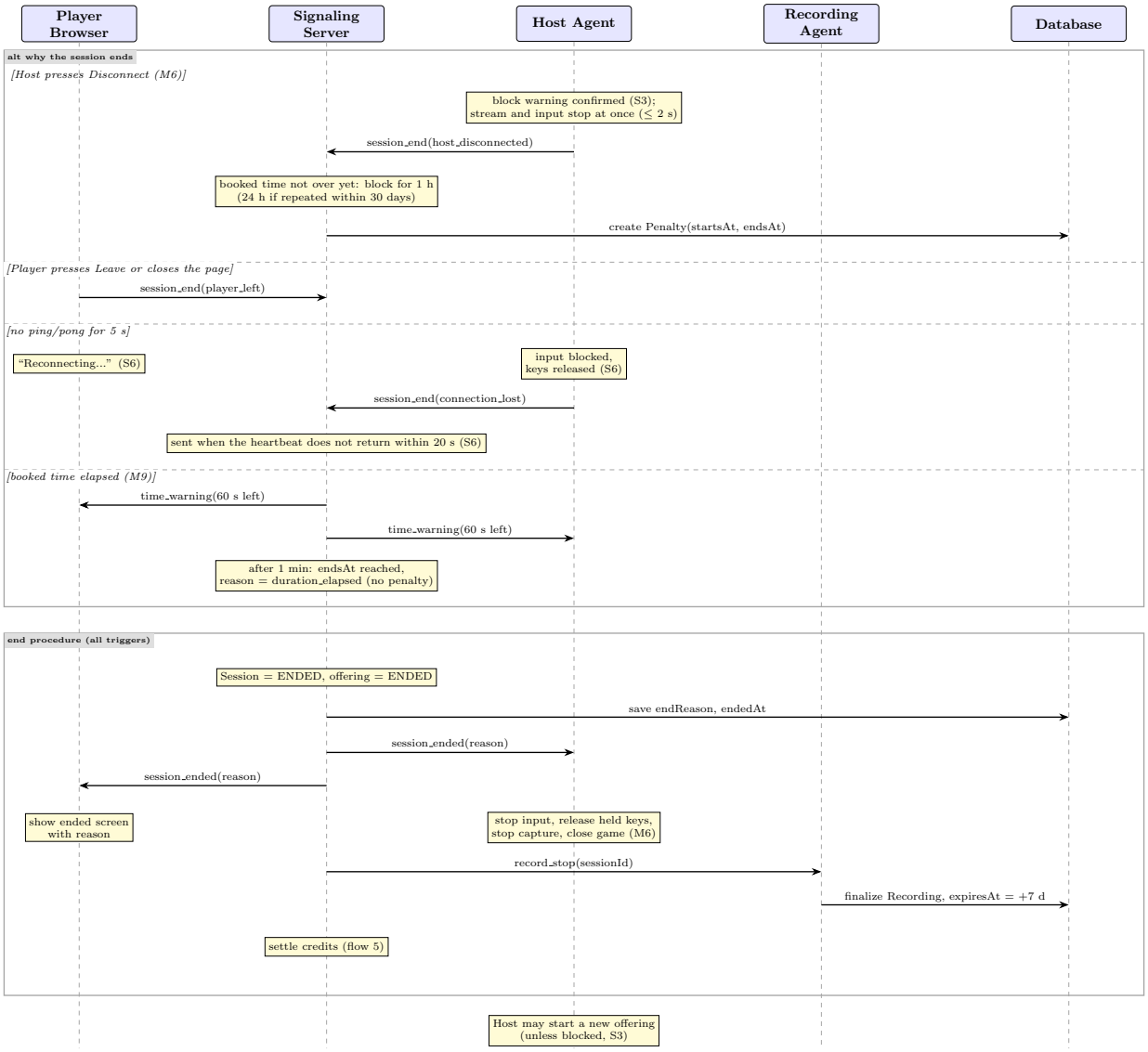


Figure 4: Session end and failure paths

Flow 5: Credits settlement (S4, G6)

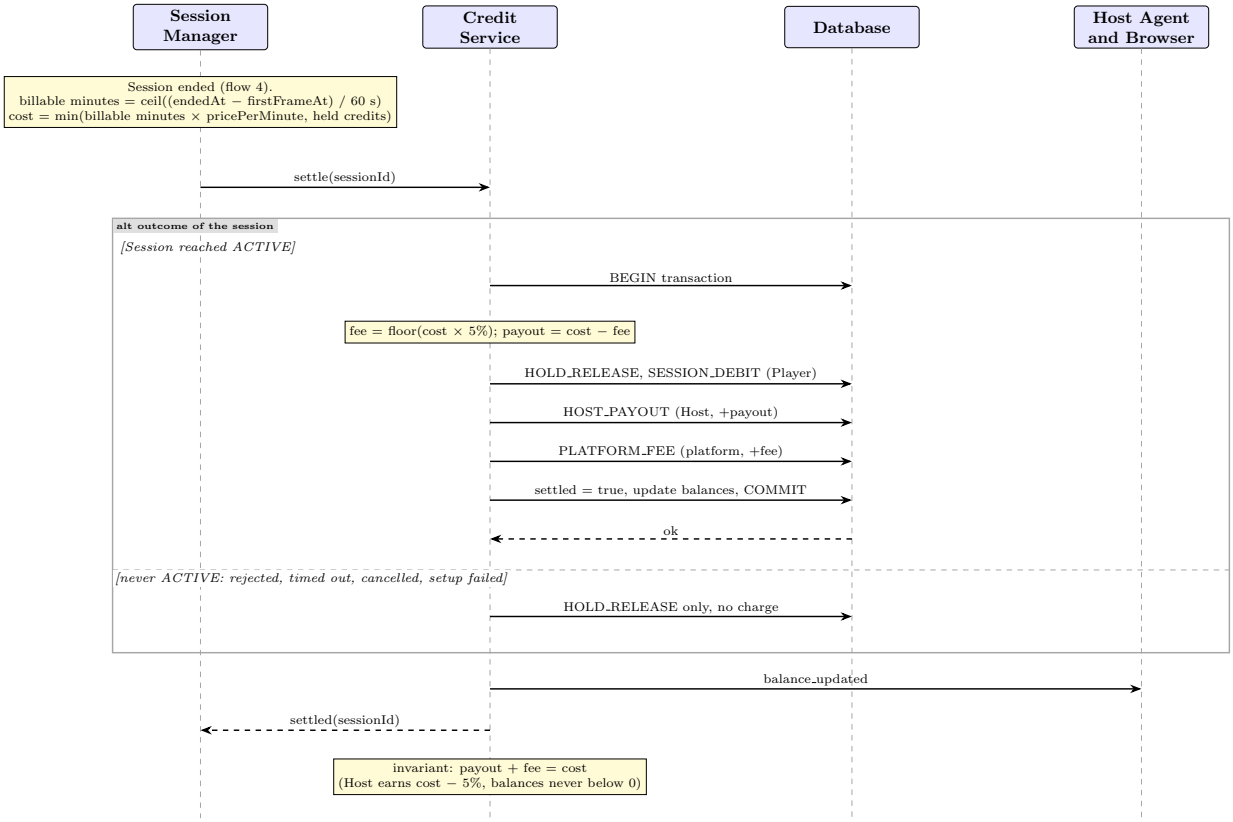


Figure 5: Credits settlement

4 Non-Functional Requirements

This section defines the quality requirements of LitasCloud: performance, reliability, security, usability, compatibility, and maintainability. Performance figures are measured on a 720p stream over a typical home connection. The last column shows the requirement or goal each item comes from.

ID	Requirement	Source
Performance		
NFR-01	The stream shall run at a 30 FPS target and shall not fall below 20 FPS during normal play.	M4, G1
NFR-02	Network round-trip latency shall be below 100 ms.	M7, G1
NFR-03	Input-to-display delay (key press to visible effect in the video) shall be below 200 ms.	M5
NFR-04	If the connection is too slow, the Host Agent shall drop video frames instead of queueing them.	M4
NFR-05	The first video frame shall appear within 10 s of Host approval (see M4).	M4, G2
NFR-06	Disconnect shall stop stream and input within 2 s (see M6).	M6, G5
Reliability		
NFR-07	A lost connection shall be detected within 5 s, and the session shall end if it does not return within 20 s (see M6, S6).	M6, S6
NFR-08	An offering shall disappear from the Player list within 10 s after the Host Agent loses its server connection (see M1).	M1
NFR-09	Reserving a Host and holding credits and settling credits, shall each be atomic (see M2, S4).	M2, S4
NFR-10	A Recording Agent failure shall not affect the Player's stream (see M13).	M13
Security		
NFR-11	Passwords shall be stored as salted hashes, traffic shall use TLS and failed logins shall be limited to 5 per minute per IP address (see M8).	M8
NFR-12	Video and input shall be encrypted in transit with DTLS/SRTP, also when relayed through TURN (see M12, S2).	M12, S2
NFR-13	Only whitelisted input shall be applied on the Host PC (see M5).	M5
Usability		
NFR-14	The Player shall need only a browser, with no installation and no plugin (see M4).	M4, G3
NFR-15	Error messages shall say what went wrong and what the user can do next.	M2, M3, S5
Compatibility		
NFR-16	The browser client shall work in the current desktop versions of Chrome, Firefox and Edge.	Section 1.5
NFR-17	The Host Agent shall run on Windows 10 and 11.	Section 1.5
Maintainability		

ID	Requirement	Source
NFR-18	The Host Agent, Signaling Server and browser client shall be separate modules that communicate only through the documented API contract.	Section 6

5 High-Level Architecture

5.1 System Overview

The system has four deployable components and three supporting services. After the setup phase, video and input travel directly between the Host Agent and the Browser Client over WebRTC. The Signaling Server never carries them (M12). The two supporting services are the PostgreSQL database and the coturn STUN/TURN server.

Table 11: Components of LitasCloud

Component	Runs on	Technology	Responsibility
Browser Client	Player's browser	HTML, CSS, vanilla JS	Login, offering list, waiting and stream screens, key and mouse capture, latency and FPS display (M2, M5, M7, M8).
Host Agent	Host's Windows 10/11 PC	Python 3.11, dxcam, aiortc	Host login, offering set-up, approval dialog, game launch on a hidden desktop, window capture, VP8 streaming, whitelisted input injection, Disconnect (M1, M3, M5, M6, M10, M11).
Signaling / API Server	Cloud (operator)	Python, FastAPI	Accounts, offerings, join requests, sessions, credits, penalties, reports. Relays SDP and ICE messages (M1, M2, M8, M12, S3, S4).
Recording Agent	Cloud (operator)	Python, aiortc	Joins an active session as an extra peer, saves a video-only copy, deletes it after 7 days (M13).
Database	Cloud (operator)	PostgreSQL	Stores the entities of Section 7: accounts, offerings, requests, sessions, credit transactions, reports, recordings, penalties.
Recording storage	Cloud (operator)	file store	Holds recording files, readable only by operators (M13).
coturn	Operator-run server	STUN / TURN	Helps the peers find a path, relays encrypted media when no direct path exists (S2).

Figure 6 shows where each component runs and how they communicate. Blue arrows are WebRTC traffic.

Component and deployment diagram

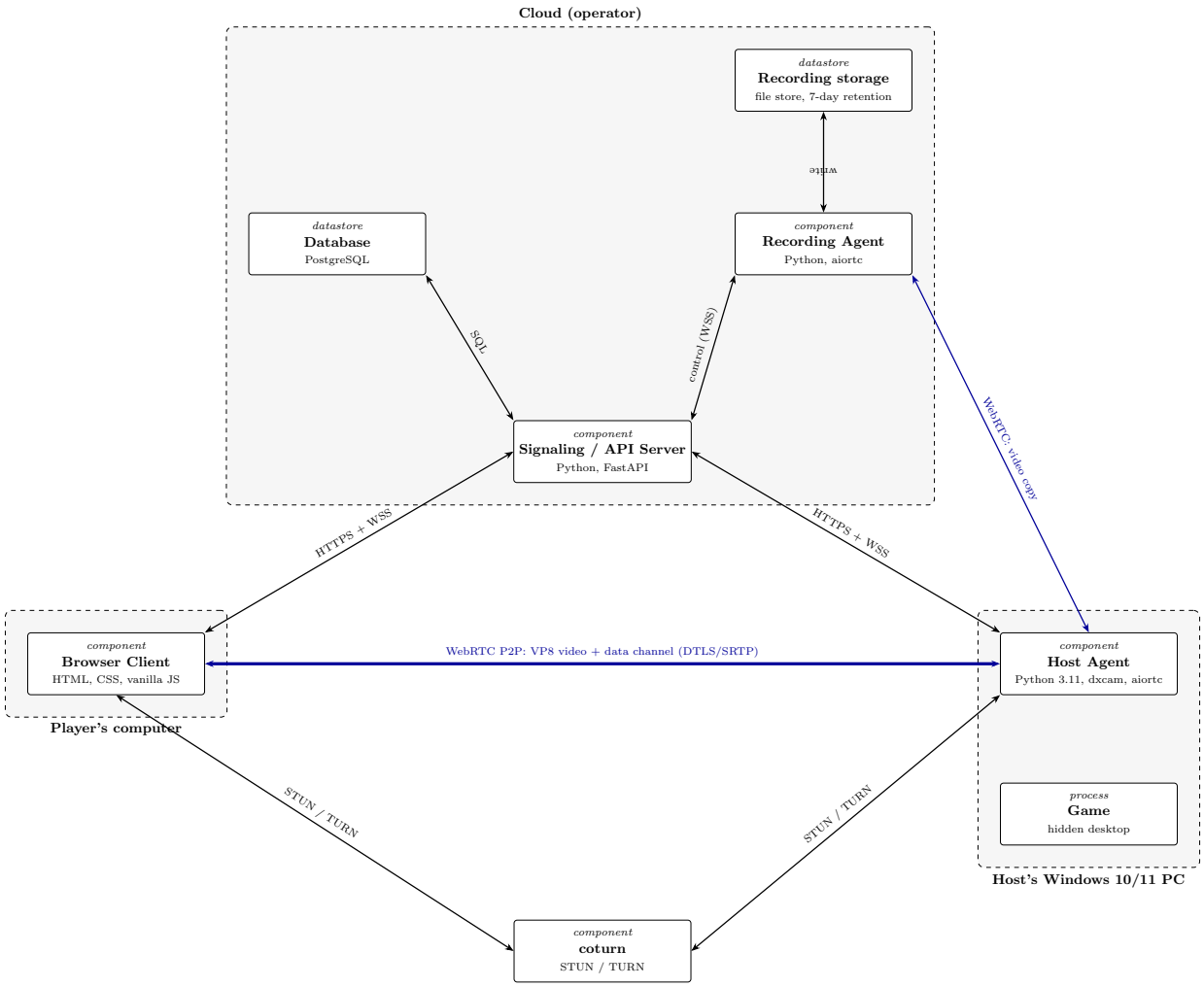


Figure 6: Component and deployment diagram

Table 12: Communication links

Link	Protocol	Content	Source
Browser Client ↔ Server	HTTPS + WSS	REST: register, login, offering list, balance, report. WebSocket: join request, SDP and ICE relay, session events.	M2, M8, M12
Host Agent ↔ Server	HTTPS + WSS	REST: login. WebSocket: register offering, heartbeat every 3 s, approval, SDP and ICE relay, session events.	M1, M3, M8
Browser Client ↔ Host Agent	WebRTC P2P (DTLS/SRTP)	VP8 video track and the data channel control (input, ping/pong).	M4, M5, M12, NFR-12
Host Agent → Recording Agent	WebRTC (DTLS/SRTP)	Copy of the video track over its own connection, video only.	M13, NFR-10

Table 12: Communication links

Link	Protocol	Content	Source
Server ↔ Recording Agent	WSS	Start and stop commands, SDP and ICE relay.	M13
Server ↔ Database	SQL	All persistent data.	M1, M2, S3, S4
Recording Agent → storage	file write	Recording files, 7-day retention.	M13
Browser Client, Host Agent ↔ coturn	STUN / TURN	Public address discovery, relay fallback.	S2
Host Agent → Game	local process	Launch on a hidden desktop, capture of the game window, injection of whitelisted input.	M5, M10, M11

Main design decisions.

- **Peer-to-peer media.** Video and input never pass through the Signaling Server, which keeps the running cost low (UN15). The only exception is the TURN relay used when no direct path exists (S2). The relayed traffic stays encrypted.
- **The server decides, the agents execute.** Accounts, reservations, credits and penalties are decided only by the Signaling Server. The Host Agent and the Browser Client hold no authority over them (M8, NFR-09).
- **Host control stays local.** Disconnect works without waiting for the server: the Host Agent stops stream and input at once, within 2 s (M6, NFR-06).
- **Recording is isolated.** The Recording Agent uses its own peer connection, so a failure there never degrades the Player's stream (M13, NFR-10).
- **Hidden game.** The game runs on a hidden desktop and only its window is captured, so the Host can keep working (M11).

5.2 Packages and Modules

There are no code dependencies between the four top-level packages. They communicate only through the API contract (NFR-18). Figure 7 shows the modules of each package. Solid arrows are dependencies inside a package, dashed blue arrows are protocol links between packages. Blue-filled boxes are the entry modules that coordinate the others.

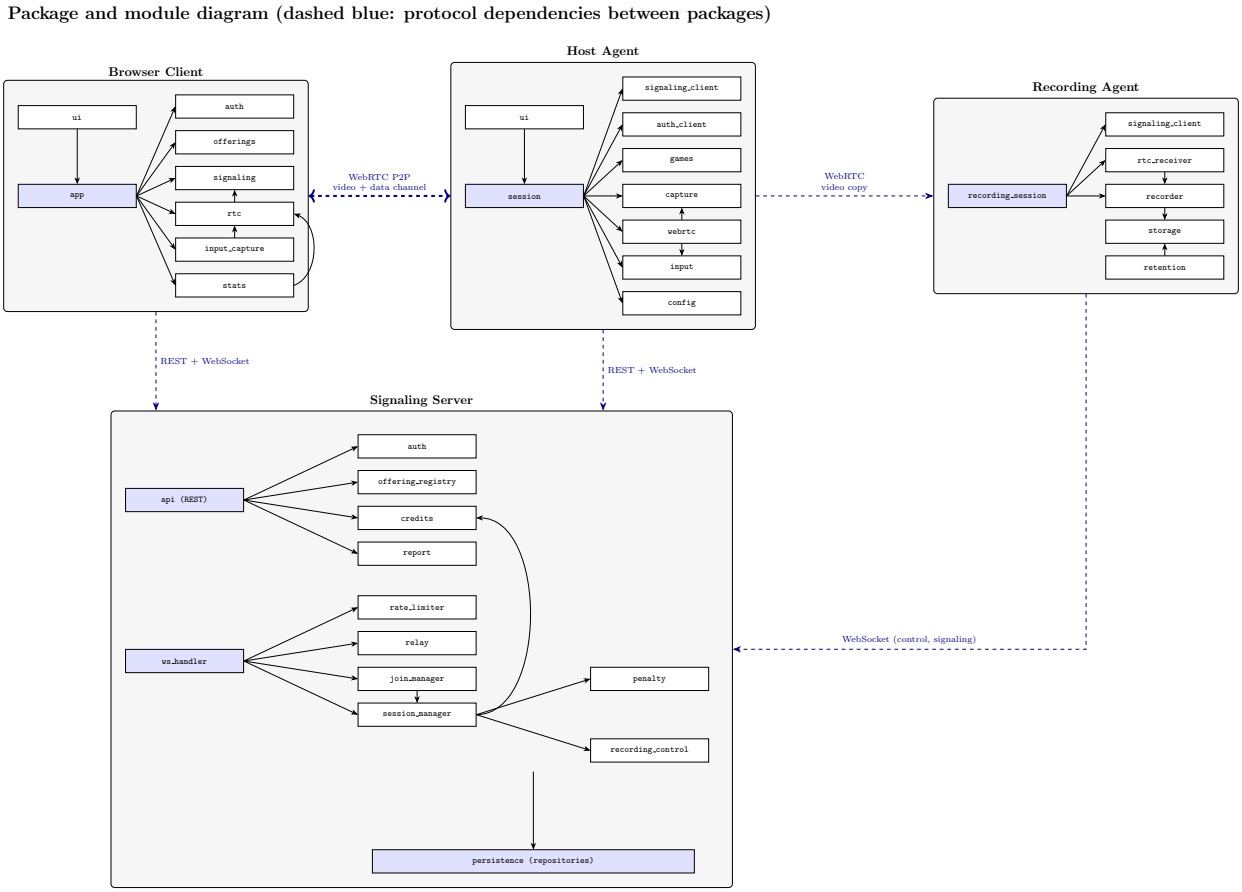


Figure 7: Package and module diagram

Browser Client

Table 13: Modules of the Browser Client

Module	Responsibility	Depends on
ui	Login, browse, waiting, stream, ended and credits screens, error messages	app
app	Client state machine and flow control	all modules below
auth	Registration and login over REST	—
offerings	Loads and filters the list of available offerings	—
signaling	WebSocket connection to the Signaling Server	—
rtc	Peer connection, video element, data channel control	signaling
input capture	Captures key and mouse events and sends them over the data channel (M5)	rtc
stats	Computes latency (ping/pong) and FPS every second (M7)	rtc

Host Agent

Table 14: Modules of the Host Agent

Module	Responsibility	Depends on
ui	Login, set-up, approval dialog, active screen with the Disconnect button	session
session	Offering and session logic, orchestrates the other modules	all modules below
signaling client	WebSocket connection to the Signaling Server, heartbeat	—
auth client	Login and token handling	—
games	Host's game list, validation, launch on a hidden desktop, closing (M10, M11)	—
capture	Captures the game window at the target FPS and resolution	—
webrtc	Peer connection, video track, data channel, frame dropping (NFR-04)	capture, input
input	Checks key events against the whitelist, injects them, releases held keys (M5)	—
config	Server URL, resolution, FPS, monitor index	—

Signaling Server

Table 15: Modules of the Signaling Server

Module	Responsibility	Depends on
api (REST)	Register, login, offering list, balance, reports	auth, offering registry, credits, report, rate limiter
ws handler	Accepts WebSocket connections, parses and routes messages	relay, join manager, session manager
auth	Password hashing, tokens, roles (M8)	—
offering registry	Registers offerings, lists available ones, reserves and releases them, expires offerings after 10 s without a heartbeat (M1)	—
credits	Holds, releases and settles credits, 5 % fee (S4)	—
report	Creates and closes reports (C7)	—
rate limiter	Limits failed logins per IP (NFR-11)	—
relay	Forwards SDP and ICE messages, only between the two participants of one session (M12)	session manager
join manager	Validates join requests, reserves the Host, runs the 30 s approval timer (M2, M3, S1)	offering registry, session manager

Table 15: Modules of the Signaling Server

Module	Responsibility	Depends on
session manager	Session state machine, end procedure, settlement trigger (M6, M9)	penalty, recording control, credits
penalty	Temporary Host block after an early Disconnect (S3)	—
recording control	Start and stop commands to the Recording Agent, session flag (M13)	—
persistence	Repositories for the database; used by all modules inside the dashed frame except rate limiter, which keeps its counters in memory	—

Recording Agent

Table 16: Modules of the Recording Agent

Module	Responsibility	Depends on
recording session	Starts and stops recording jobs on server commands	signaling client, rtc receiver, recorder
signaling client	WebSocket connection to the Signaling Server	—
rtc receiver	Receives the copy of the video track	recorder
recorder	Writes the received frames to a file	storage
storage	Stores, reads and deletes recording files	—
retention	Deletes recordings after 7 days, unless a report is open (M13, C7)	storage

5.3 Class Diagrams

The diagrams show the main classes of each component. Arrows point from the class that uses to the class that is used. Attribute and method lists are simplified.

Host Agent

Class diagram: Host Agent

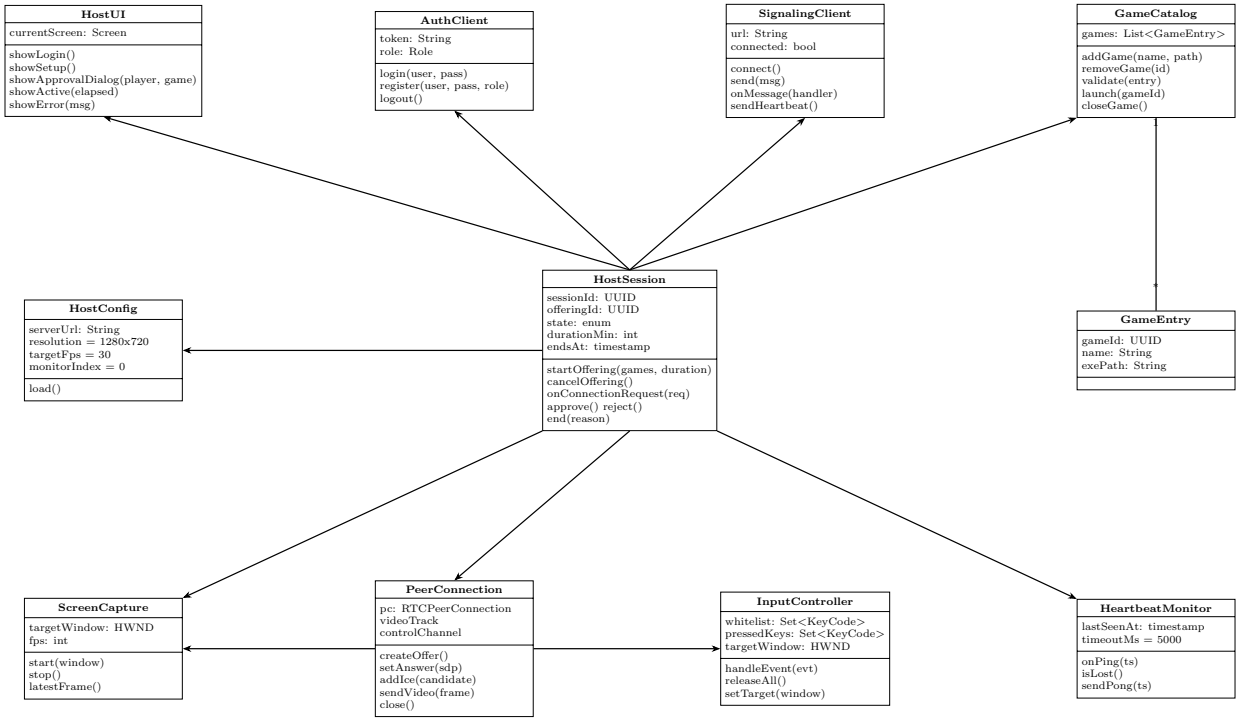


Figure 8: Class diagram: Host Agent

`HostSession` owns the offering and session logic and uses all other classes. `PeerConnection` takes frames from `ScreenCapture`, sends them as the video track and passes input events from the data channel to `InputController`. `InputController` applies only whitelisted keys and mouse actions while the game window is the target, and releases all held keys when the session ends (M5, M6). `HeartbeatMonitor` answers the browser's ping with a pong and detects a lost connection after 5 s (NFR-07). `GameCatalog` launches only games from the Host's own list (M10).

Signaling Server

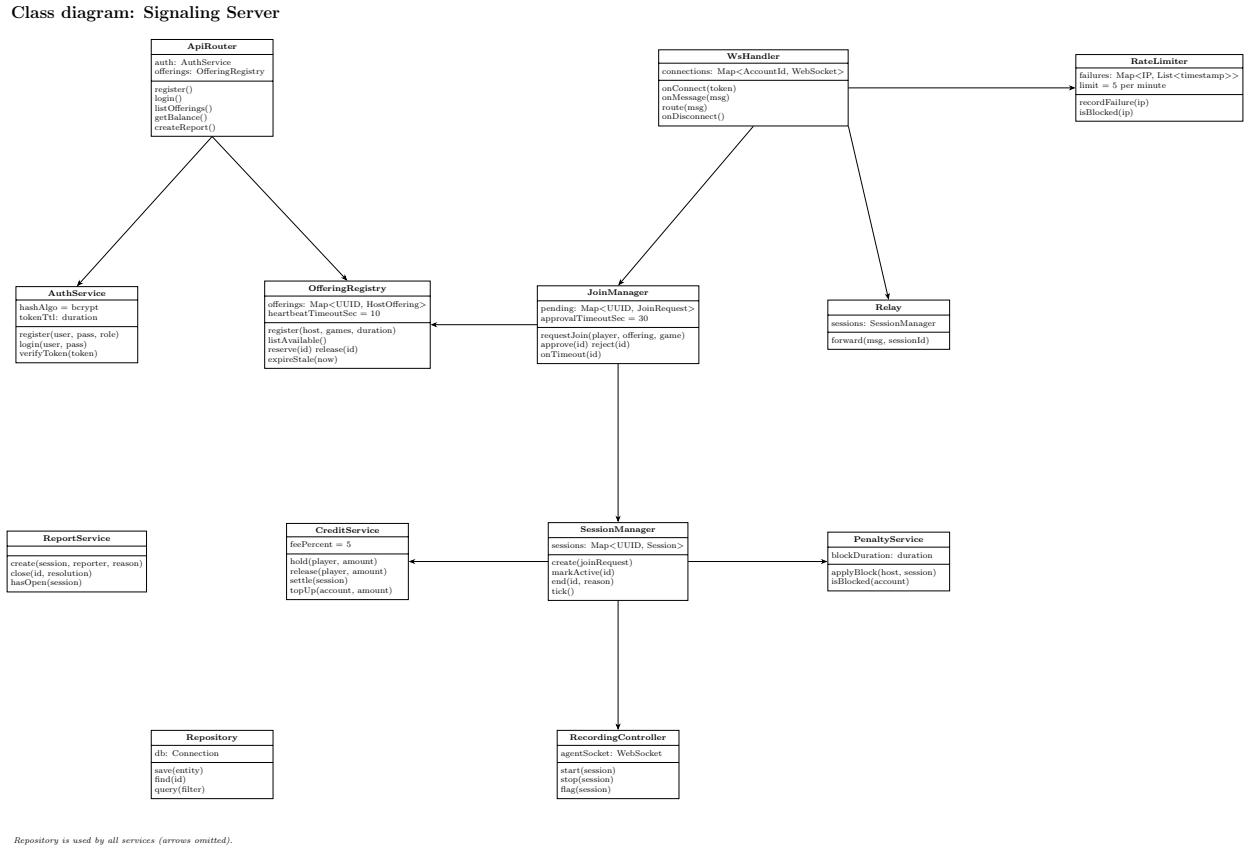


Figure 9: Class diagram: Signaling Server

`ApiRouter` serves the REST requests and uses `RateLimiter` to limit failed logins. `WsHandler` receives WebSocket messages and routes them to `JoinManager`, `Relay` and `SessionManager`. `JoinManager` asks `OfferingRegistry` to reserve the Host and `SessionManager` to create the session once the Host approves. `SessionManager` owns the session state machine (Section 5.4) and on session end calls `CreditService` (settlement), `PenaltyService` (early Disconnect) and `RecordingController`. Reserving a Host together with holding credits, and settling credits, are each one atomic database operation (NFR-09). `Repository` is used by all services (arrows omitted).

Browser Client

Class diagram: Browser Client

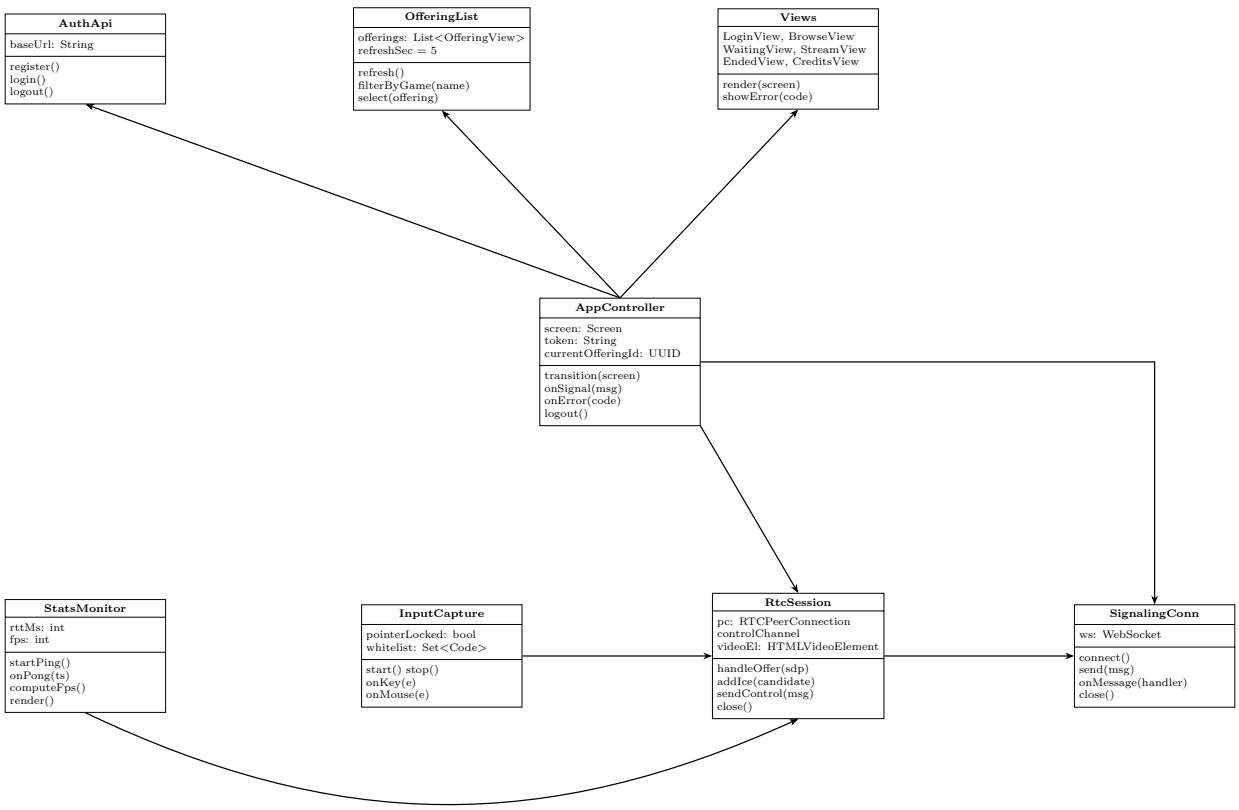


Figure 10: Class diagram: Browser Client

`AppController` holds the current screen and token and reacts to server messages. `RtcSession` owns the peer connection, the video element and the data channel `control`. `InputCapture` sends whitelisted key and mouse events through it, and `StatsMonitor` measures latency and FPS every second (M7). `SignalingConn` carries all messages to the Signaling Server.

Recording Agent

Class diagram: Recording Agent

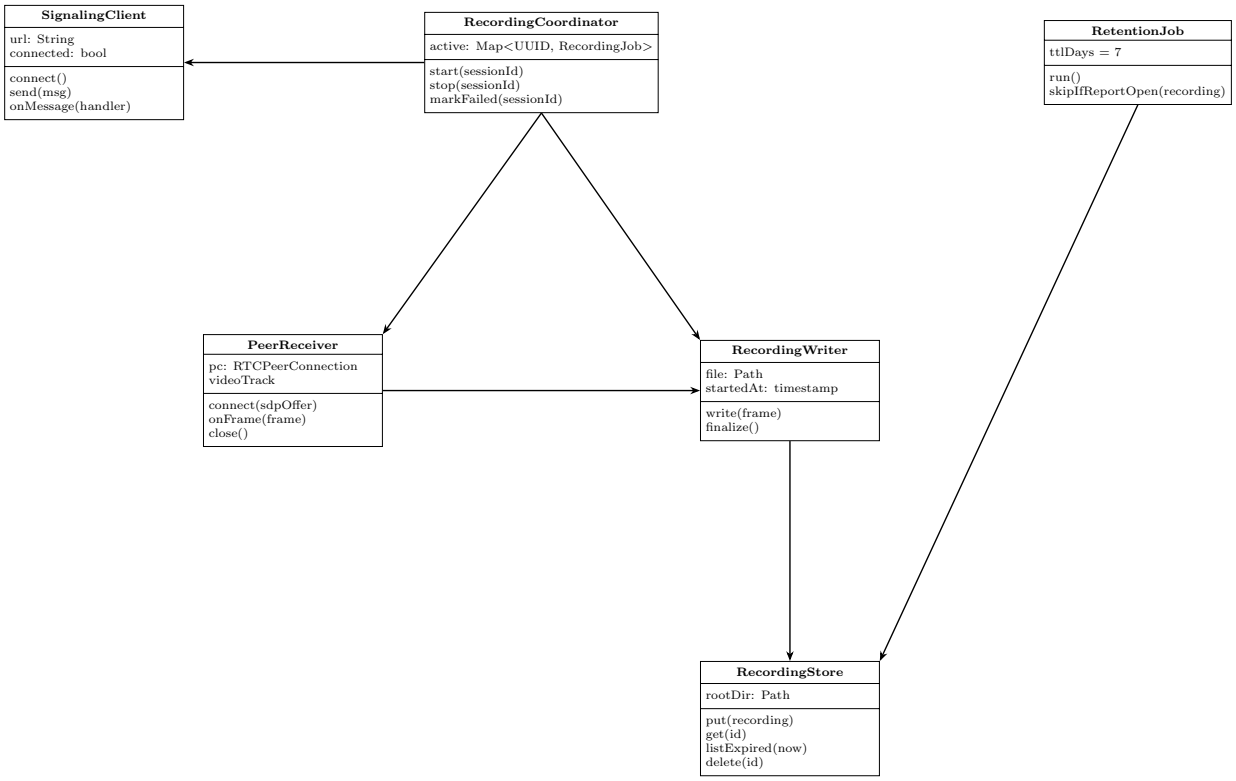


Figure 11: Class diagram: Recording Agent

`RecordingCoordinator` reacts to start and stop commands of the Signaling Server. It creates a `PeerReceiver` for the copy of the video track and a `RecordingWriter` that writes frames to a file in `RecordingStore`. If writing fails, the coordinator reports it to the server, which flags the session; the Player's stream is not affected (NFR-10). `RetentionJob` removes recordings that are older than 7 days, unless a report is open (M13, C7).

5.4 State Machines

Two state machines drive the service (Figure 12). Both are owned by the Signaling Server, so the Host Agent and the Browser Client only show the state. `HostOffering.status` describes the offer of a Host and `Session.state` describes one play session. The end reasons of a session are listed in Table 9.

State machines: offering and session lifecycle

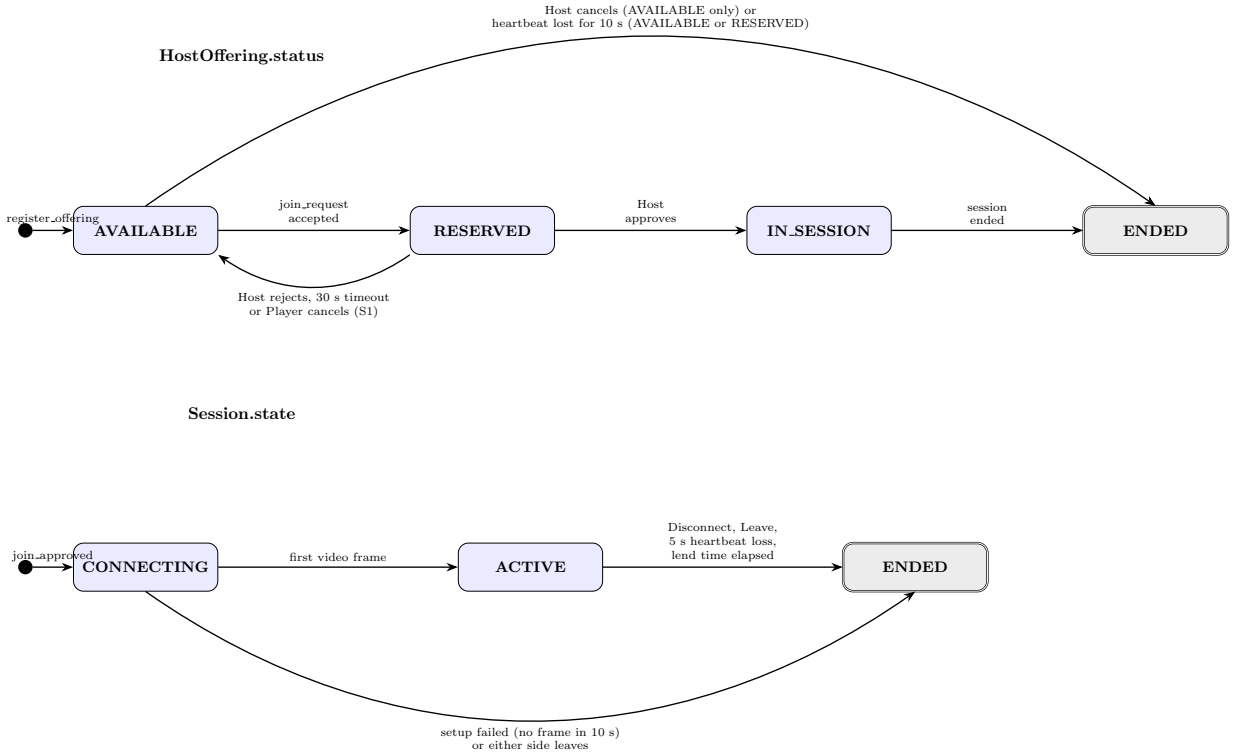


Figure 12: State machines: offering and session lifecycle

Table 17: Transitions of HostOffering.status

From	To	Trigger	Effect
(start)	AVAILABLE	register offering accepted (M1)	Offering is visible in the Player list.
AVAILABLE	RESERVED	join request accepted (M2)	Host reserved and Player's credits held in one atomic step. Request is PENDING.
RESERVED	IN SESSION	Host approves (M3)	Request APPROVED, session created in CONNECTING.
RESERVED	AVAILABLE	Host rejects, 30 s timeout, or Player cancels (M3, S1)	Hold released. Request REJECTED, TIMED OUT or CANCELLED.
AVAILABLE	ENDED	Host cancels, offered time over, or no heartbeat for 10 s (M1)	Offering removed from the list.
RESERVED	ENDED	No heartbeat for 10 s (M1)	Waiting request rejected, hold released.
IN SESSION	ENDED	Session ended	Host may start a new offering, unless blocked (S3).

Table 18: Transitions of Session.state

From	To	Trigger	Effect
(start)	CONNECTING	Host approves, join approved (M3)	Game launched, SDP and ICE exchange starts.
CONNECTING	ACTIVE	First video frame shown in the browser (M4, M9)	Booked time and billing start counting.
CONNECTING	ENDED	No frame within 10 s of approval, or either side leaves (M4, G2)	End reason setup failed, Player is not charged.
ACTIVE	ENDED	Host Disconnect, Player Leave, connection lost (no heartbeat for 5 s and not back within 20 s), or booked time elapsed (M6, M9, S6)	End procedure (Section 3.4), credit settlement (Section 3.5).

6 API Contract

The API contract is defined in **litascloud-api.yaml**, located in the project's GitHub repository. To view the specification, navigate to Swagger Editor (www.editor.swagger.io), select File > Import File, and upload the YAML file.

7 Data Model

The Signaling Server keeps all persistent data in one PostgreSQL database (see Table 11). The Host Agent, Browser Client and Recording Agent have no database of their own; the only local data is the configuration file of the Host Agent (Table 33). Identifiers are UUIDs, timestamps are UTC (`timestamptz`), credits are whole numbers (W3), and enumerations are PostgreSQL enum types. A field is mandatory unless it is marked *nullable*. Figure 13 shows the ten tables with their fields and data types and the relationships between them. The following subsections list the enumerations, every entity, the relationships, the integrity rules and the values that are computed or kept only in memory.

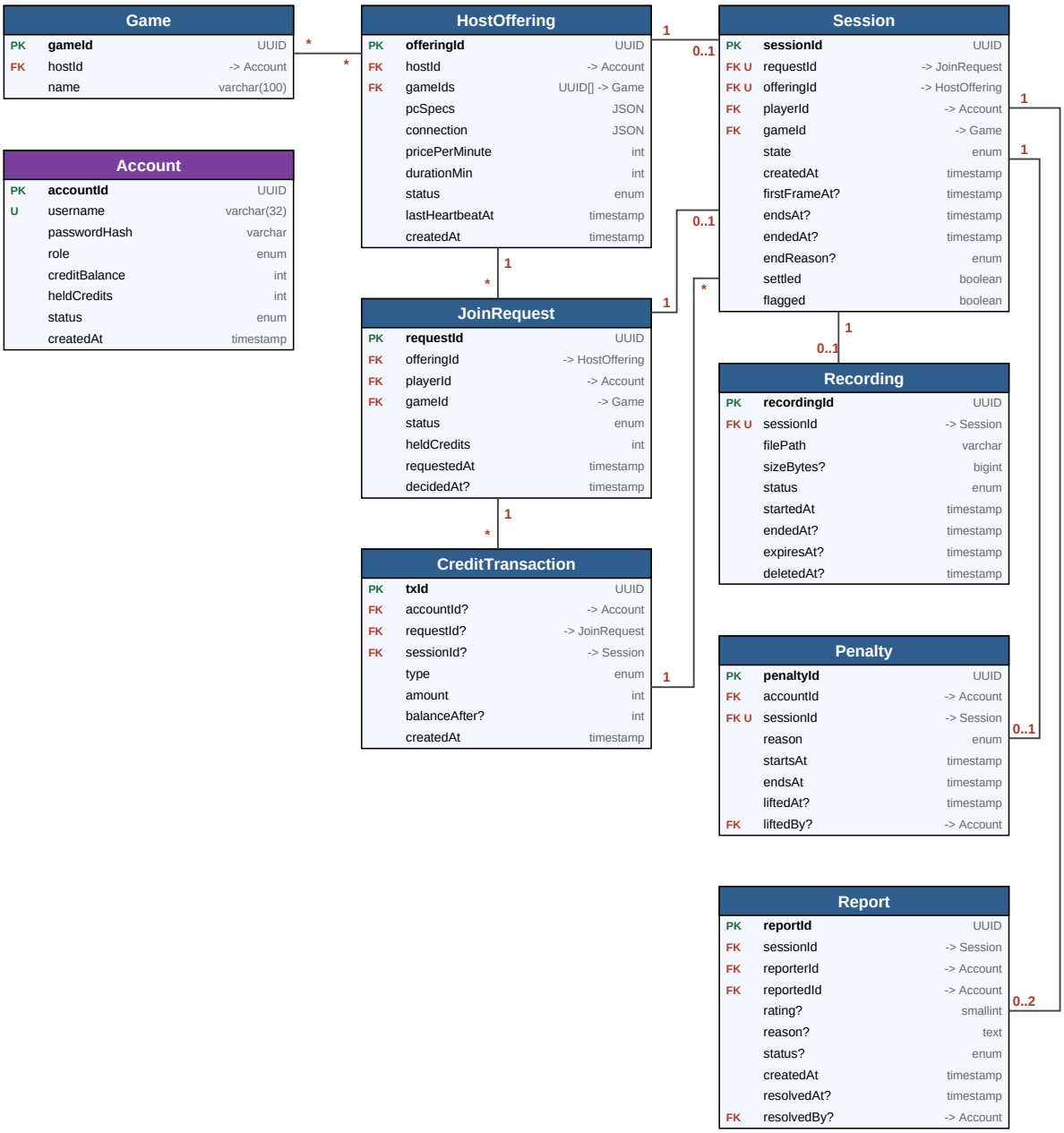


Figure 13: Data model (PostgreSQL)

7.1 Enumerations

Table 19 lists every enumeration used by the entities, with its values and the requirement, table or API schema it comes from. The values are the same as in the API contract and in the state machines of Section 5.4.

Table 19: Enumerations

Enumeration	Values	Source
Role	`HOST`, `PLAYER`, `OPERATOR`. Operator accounts are created by the operator, never by `POST /auth/register`.	M8, API
AccountStatus	`ACTIVE`, `DISABLED`	API
OfferingStatus	`AVAILABLE`, `RESERVED`, `IN_SESSION`, `ENDED`	Table 17
RequestStatus	`PENDING`, `APPROVED`, `REJECTED`, `TIMED_OUT`, `CANCELLED`	Section 3.2
SessionState	`CONNECTING`, `ACTIVE`, `ENDED`	Table 18
EndReason	`host_disconnected`, `player_left`, `connection_lost`, `duration_elapsed`, `setup_failed`	Table 9
TransactionType	`TOP_UP`, `HOLD`, `HOLD_RELEASE`, `SESSION_DEBIT`, `HOST_PAYOUT`, `PLATFORM_FEE`	S4
ReportStatus	`OPEN`, `RESOLVED`, `DISMISSED`	C7
RecordingStatus	`RECORDING`, `FINALIZED`, `FAILED`	M13, API
PenaltyReason	`HOST_ENDED_EARLY`	S3

A join request that ends because the Host Agent went offline gets the status `REJECTED` (API reason `HOST_UNAVAILABLE`).

7.2 Entities

Account. Every person who uses the service has one account with one role (M8). The account also holds the credit balance, split into available and held credits (S4). A disabled account cannot log in or act. Operator accounts are only used to review reports and lift blocks.

Table 20: Account: a Host, a Player or an Operator (M8)

Field	Type	Constraints / description
accountId	UUID	Primary key.
username	varchar(32)	Unique. 3–32 letters, digits or underscores.
passwordHash	varchar	Salted bcrypt hash, never the plain password (NFR-11).
role	Role	Fixed at registration.
creditBalance	int	≥ 0. Available credits (`available` in `GET /balance`). Held credits are not included.
heldCredits	int	≥ 0. Credits held for a pending or active request (S4).
status	AccountStatus	`DISABLED` blocks login and all actions.
createdAt	timestamp	Time of registration.

Game. A Host adds games to the Host Agent by name and program path (M10). Only the identifier and the name are sent to the server, so Players can see which games a PC offers. The program path stays on the Host PC.

Table 21: Game: a game that a Host added to the Host Agent (M10)

Field	Type	Constraints / description
gameId	UUID	Primary key. Created by the Host Agent and sent in `register_offering`.
hostId	UUID	Foreign key to Account (role `HOST`).
name	varchar(100)	1–100 characters, for example “Minecraft”. The program path is never sent to the server (M10).

HostOffering. When a Host presses “Start hosting”, the server stores one offering with the chosen games, the duration, the PC specifications and the self-test result (M1, S5, C5). The offering moves through the states of Table 17. It is used for at most one session. Afterwards it is `ENDED` and the Host creates a new one.

Table 22: HostOffering: one offer of a Host PC (M1)

Field	Type	Constraints / description
offeringId	UUID	Primary key.
hostId	UUID	Foreign key to Account.
gameIds	UUID[]	1–50 distinct games of this Host (M10). PostgreSQL cannot enforce a foreign key on an array, so the server checks it on registration.
pcSpecs	JSON	`cpu`, `gpu`, `ramGb`. Read by the Host Agent (C5).
connection	JSON	`uploadMbps` (≥ 4) and `latencyMs`. Result of the self-test (S5, C5).
pricePerMinute	int	≥ 1 credits. Set by the server (S4).
durationMin	int	15–240 (M9).
status	OfferingStatus	See Table 17.
lastHeartbeatAt	timestamp	Updated every 3 s. After 10 s without an update the offering becomes `ENDED` (M1).
createdAt	timestamp	Time of registration.

JoinRequest. It is created when a Player presses Play. It records the chosen offering and game and the credits that were held for the offered duration (M2, S4). It stays `PENDING` until the Host approves or rejects, the 30 s timer fires, or the Player cancels (M3, S1).

Table 23: JoinRequest: a Player’s request to use an offering (M2, M3, S1)

Field	Type	Constraints / description
requestId	UUID	Primary key.
offeringId	UUID	Foreign key to HostOffering.
playerId	UUID	Foreign key to Account.
gameId	UUID	Foreign key to Game. Must be in the offering’s `gameIds`.
status	RequestStatus	`PENDING` until decided.

Field	Type	Constraints / description
heldCredits	int	≥ 1. `pricePerMinute` $\times$ `durationMin` of the offering (S4).
requestedAt	timestamp	The 30 s approval timer starts here (M3).
decidedAt	timestamp	Nullable. Set on approval, rejection, timeout or cancellation.

Session. A session is created in state `CONNECTING` when the Host approves a request (Section 3.3). It becomes `ACTIVE` at the first video frame, which also starts the booked time and the billing (M9, S4). When it ends, the end reason and time are stored and the credits are settled once (Section 3.5).

Table 24: Session: one play session

Field	Type	Constraints / description
sessionId	UUID	Primary key.
requestId	UUID	Foreign key to JoinRequest. Unique.
offeringId	UUID	Foreign key to HostOffering. Unique, because an offering has at most one session. The Host is `HostOffering.hostId`.
playerId	UUID	Foreign key to Account.
gameId	UUID	Foreign key to Game.
state	SessionState	See Table 18.
createdAt	timestamp	Time of approval. The 10 s setup timer runs from here (M4, G2).
firstFrameAt	timestamp	Nullable until `ACTIVE`. Billing and the booked time start here (M9, S4).
endsAt	timestamp	Nullable until `ACTIVE`. `firstFrameAt` + `durationMin`.
endedAt	timestamp	Nullable until `ENDED`.
endReason	EndReason	Nullable until `ENDED` (Table 9).
settled	boolean	Default false. Set to true in the settlement transaction (Section 3.5).
flagged	boolean	Default false. True if recording failed (M13).

CreditTransaction. Every movement of credits is written as one row and rows are never changed or deleted, so balances can always be checked against the ledger (S4, G6). Table 26 explains the six transaction types.

Table 25: CreditTransaction: append-only ledger of every credit movement (S4)

Field	Type	Constraints / description
txId	UUID	Primary key.
accountId	UUID	Nullable foreign key to Account. Null only for `PLATFORM_FEE`.

Field	Type	Constraints / description
requestId	UUID	Nullable foreign key to JoinRequest. Set for every type except `TOP_UP`, because a hold is created before any session exists.
sessionId	UUID	Nullable foreign key to Session. Set once a session exists.
type	Transaction-Type	See Table 26.
amount	int	Signed, never 0.
balanceAfter	int	Nullable. `creditBalance` of the account after the entry. Null for `PLATFORM_FEE`.
createdAt	timestamp	Time of the entry.

A hold moves credits from the available balance to the held amount when a request is made. At settlement the hold is released and the real cost is charged, so the Player pays only for the time played (S4). The Host receives the cost minus the 5% fee, and the fee is booked to the platform.

Table 26: Credit transaction types

Type	Account	Amount	Effect
TOP_UP	Player	+	`creditBalance` increases (prototype helper).
HOLD	Player	−	`creditBalance` decreases and `heldCredits` increases by the same amount.
HOLD_RELEASE	Player	+	Reverse of the hold: `creditBalance` increases and `heldCredits` decreases.
SESSION_DEBIT	Player	−	The cost of the session.
HOST_PAYOUT	Host	+	Cost minus fee.
PLATFORM_FEE	none	+	$\lfloor 5\% \times \text{cost} \rfloor$.

Report. After a session has ended, each side can rate the other (1–5) and optionally describe a problem (C7). A report with a reason links to the session recording and keeps it past the 7-day deletion while the report is open (M13).

Table 27: Report: a rating and/or a problem report after a session (C7)

Field	Type	Constraints / description
reportId	UUID	Primary key.
sessionId	UUID	Foreign key to Session. The session must be `ENDED`.
reporterId	UUID	Foreign key to Account, a participant of the session. Unique together with `sessionId`, so each side submits once.
reportedId	UUID	Foreign key to Account. The other participant.
rating	smallint	Nullable. 1–5.
reason	text	Nullable. 1–1000 characters.

Field	Type	Constraints / description
status	ReportStatus	Nullable. Null if only a rating was given, otherwise starts as `OPEN`. It is null exactly when `reason` is null.
createdAt	timestamp	Time of submission.
resolvedAt	timestamp	Nullable.
resolvedBy	UUID	Nullable foreign key to Account (Operator).

At least one of `rating` and `reason` is set.

Recording. The Recording Agent saves a video-only copy of every active session (M13). The row describes the file. If recording fails, the session continues and is flagged (NFR-10). The retention job deletes the file after 7 days unless a report is open.

Table 28: Recording: the saved video copy of a session (M13)

Field	Type	Constraints / description
recordingId	UUID	Primary key.
sessionId	UUID	Foreign key to Session. Unique.
filePath	varchar	Path in the recording storage.
sizeBytes	bigint	Nullable until `FINALIZED`.
status	RecordingStatus	If `FAILED`, `Session.flagged` is true and the session is not affected (NFR-10).
startedAt	timestamp	Time the recording started.
endedAt	timestamp	Nullable.
expiresAt	timestamp	Nullable until `FINALIZED`. `endedAt` + 7 days. The recording is kept while an `OPEN` report exists for the session (C7).
deletedAt	timestamp	Nullable. Set when the retention job deletes the file. The row is kept.

Penalty. If a Host presses Disconnect before the booked time ends, the server stores a temporary block (S3). The block is 1 hour, or 24 hours if the Host already had a penalty within the last 30 days. An Operator can lift it after review.

Table 29: Penalty: temporary Host block after an early Disconnect (S3)

Field	Type	Constraints / description
penaltyId	UUID	Primary key.
accountId	UUID	Foreign key to Account (the Host).
sessionId	UUID	Foreign key to Session. Unique.
reason	PenaltyReason	Why the block was applied.
startsAt	timestamp	Time the session ended.
endsAt	timestamp	`startsAt` + 1 h, or + 24 h if the Host already has a penalty that started within the last 30 days.
liftedAt	timestamp	Nullable. Set when an Operator lifts the block (S3).

Field	Type	Constraints / description
liftedBy	UUID	Nullable foreign key to Account (Operator).

A Host is blocked while the current time is between `startsAt` and `endsAt` and `liftedAt` is null.

7.3 Relationships

Table 30 describes the relationships drawn in Figure 13. One Account appears in several roles: as Host it owns games, offerings and penalties, as Player it owns join requests and sessions. The Host of a session is found through its offering.

Table 30: Relationships between entities

From	To	Meaning
Account (Host)	Game, HostOffering	1 to many.
Account (Host)	Penalty	1 to many.
Account (Player)	JoinRequest, Session	1 to many.
Account	CreditTransaction	1 to many. The platform fee has no account.
Account	Report	1 to many, as reporter and as reported.
HostOffering	JoinRequest	1 to many. A rejected or cancelled request returns the offering to `AVAILABLE`.
HostOffering	Session	1 to 0..1.
HostOffering	Game	Many to many through `gameIds`.
JoinRequest	Session	1 to 0..1. Only an approved request gets a session.
JoinRequest	CreditTransaction	1 to many.
Session	CreditTransaction	1 to many.
Session	Recording, Penalty	1 to 0..1.
Session	Report	1 to 0..2, one per participant.

7.4 Integrity rules

Table 31 lists the rules that the database or the server enforces so that the data stays consistent. They implement the requirements named in the first column, in particular that two Players can never reserve the same Host and that credits are never lost or created.

Table 31: Integrity rules

Rule	Implementation
One offering per Host (M1)	Partial unique index on `HostOffering(hostId)` where `status` is `AVAILABLE`, `RESERVED` or `IN_SESSION`.

Rule	Implementation
One request per Player (M2)	Partial unique index on `JoinRequest(playerId)` where `status` is `PENDING`. An active session of the same Player is checked in the reservation transaction, which locks the Player's account row.
Atomic reservation (NFR-09)	Changing the offering from `AVAILABLE` to `RESERVED`, creating the `PENDING` request and writing the `HOLD` entry with the account update happen in one transaction. The offering is changed with a conditional update (`WHERE status = 'AVAILABLE'`), so two Players can never reserve the same Host.
Atomic settlement (NFR-09)	`HOLD_RELEASE`, `SESSION_DEBIT`, `HOST_PAYOUT`, `PLATFORM_FEE` and `settled = true` are written in one transaction, and only if `settled` is false.
Balances	`CHECK` constraints: `creditBalance` ≥ 0 and `heldCredits` ≥ 0. `heldCredits` equals the held credits of the Player's pending or unsettled active request, or 0.
Credit conservation (G6)	For every settled session, `HOLD` and `HOLD_RELEASE` add up to 0, and `SESSION_DEBIT`, `HOST_PAYOUT` and `PLATFORM_FEE` together add up to 0.
Value ranges	`durationMin` 15–240 (M9), `pricePerMinute` ≥ 1, `rating` 1–5.
Retention (M13)	The retention job deletes the file of a recording whose `expiresAt` has passed and that has no `OPEN` report, and sets `deletedAt`. Rows are never deleted, so history and the ledger stay complete.

7.5 Derived values and data outside the database

Some values that appear in the API are not stored but calculated from the tables above, and some short-lived data is kept only in the memory of the Signaling Server. Table 32 lists both. The in-memory data is lost on a restart, which is acceptable because each item is short-lived.

Table 32: Values that are computed or kept in memory

Value	Definition
hostRating, playerRating	Average `Report.rating` where the account is `reportedId`. Null if there is none. Computed, not stored.
playedSeconds	`endedAt` − `firstFrameAt`, or 0 if the session never became `ACTIVE`. Computed.
creditsDelta	Sum of the account's `SESSION_DEBIT` and `HOST_PAYOUT` entries for the session. Computed.
Billable minutes, cost	Calculated at settlement as in Section 3.5. The results are stored in the ledger.
Failed-login counters	Kept in memory per IP address (NFR-11). Lost on restart.
WebSocket connections, timers	Open connections (at most one per account) and the 30 s approval, 10 s setup, 10 s heartbeat and 20 s reconnection timers. In memory, lost on restart.
Access tokens	Signed JWTs valid for 24 hours. Not stored.

7.6 Local data of the Host Agent

The Host Agent keeps its settings and its game list in a local configuration file on the Host PC (Table 33). Only the `gameId` and `name` of each game are sent to the server (M10).

Table 33: HostConfig: local settings of the Host Agent (configuration file)

Field	Type	Constraints / description
serverUrl	string	Address of the Signaling Server.
resolution	string	Default `1280x720`.
targetFps	int	Default 30.
monitorIndex	int	Default 0 (primary monitor).
games	list of GameEntry	`gameId` (UUID), `name`, `exePath`. Only `gameId` and `name` are sent to the server (M10). The path stays on the Host PC.

8 UI/UX Wireframes

8.1 Player Wireframes

[Link to Wireframes in Figma]

This subsection specifies every screen of the Browser client Player uses, Host application and Operator UI. The wireframes are low-fidelity: they fix the content, the controls, the states and the transitions of each screen, not colours, fonts or exact pixel sizes. The Operator console is specified in Section 8.2 and the Host Agent screens in Section 8.3. The Player screens cover the whole visit: signing in or registering, browsing the available games, waiting for the Host, connecting, playing, and the result of the visit.

Conventions used by all wireframes. Every screen is drawn inside a browser window whose address bar reads “LitasCloud — Cloud gaming”. Under each wireframe a one-line note lists the requirements it implements (for example “M3, S1, S4”). A screen that exists in several states is drawn once per state, and the states share the same identifier. The identifiers are: A1 Sign in, A2 Create account, P1 Browse available games, P2 Waiting for Host approval, P3 Connecting to the game, P4 Play the streamed game (with the one-minute warning state P4a), P5 Recovering a dropped connection and P6 Result (seven variants). There are no pairing codes and no connection tokens: the Player never types a code, and access is decided only by the authenticated accounts and the state kept by the Signaling Server (M12). For this reason every result screen returns to the game catalog (P1) and never to a code-entry screen.

Screen overview. Table 34 lists all Player screens and their main purpose.

Table 34: Overview of the Player screens

ID	**Screen**	**Purpose and requirements**
A1	Sign in	Log in with username and password (M8).
A2	Create account	Register a Host or Player account (M8).

ID	Screen	Purpose and requirements
P1	Browse available games	List available offers with duration, price and PC specifications, and send a join request (M2, M9, S4, C5, M13).
P2	Waiting for Host approval	Wait for the Host's decision for at most 30 s, with a Cancel button (M2, M3, S1, S4).
P3	Connecting to the game	Set up the peer connection until the first video frame arrives, including the relay state (M4, M9, S2).
P4, P4a	Play the streamed game, one-minute warning	Show the video, take keyboard and mouse input, show remaining time, latency and FPS (M4–M7, M9, M13).
P5	Recovering a dropped connection	Pause the controls and try to recover for up to 20 s (S6).
P6	Result (7 variants)	Explain why the visit ended and what it cost (M3, M4, M6, M9, S4).

Screen flow. Table 35 lists the transitions between the Player screens. The Signaling Server owns all states (Section 5.4), so the Browser Client changes screen only when it receives the message named in the table or when the Player presses the control named in the table.

Table 35: Transitions between the Player screens

From	Trigger	To
A1	Sign in accepted for a Player account	P1
A1	"Create account" pressed	A2
A2	"Sign in" link pressed	A1
P1	"Play" pressed and `request_pending` received	P2
P2	`join_approved` received	P3
P2	`join_rejected` received because the Host rejected	P6 Request declined
P2	`join_rejected` received because the 30 s timer fired	P6 Host did not respond
P2	"Cancel request" pressed and `request_cancelled` received	P1
P3	First video frame shown	P4
P3	`session_ended` (`setup_failed`) received	P6 The game could not connect
P4	`time_warning` (60 s left) received	P4a
P4, P4a	No heartbeat answer for 5 s	P5
P5	Heartbeat answered again within 20 s	P4 (or P4a)
P4, P4a, P5	"Leave" pressed	P6 You left the session
P4, P4a	`session_ended` (`host_disconnected`) received	P6 The Host ended the session
P4, P4a	`session_ended` (`duration_elapsed`) received	P6 Session complete
P5	`session_ended` (`connection_lost`) received	P6 Connection lost
P6 (all)	"Back to games" pressed	P1

Shared header. All screens after sign in (P1 to P6) show the same header: the "LitasCloud" name on the left, the Player's username and credit balance in the middle-right, and the "Sign out" button on the right, with a line below. A1 and A2 show only the "LitasCloud" name. The balance text depends on the state of the Player's credits (S4), as listed in Table 36. The number is the field `creditBalance` of the account (Table 20); held credits are not included in it.

Table 36: States of the credit display in the header

State	Header text	Screens
No credits are held	"jonas20 | 180 credits"	P1, P6 Request declined, Host did not respond, The game could not connect
Credits are held for a pending request or an active session	"jonas20 | 120 available credits"	P2, P3, P4, P4a, P5
Session ended after play, settlement not finished yet	Username only ("jonas20")	P6 You left the session, The Host ended the session, Connection lost
Settlement finished	"jonas20 | 120 credits", the new balance	P6 Session complete

8.1.1 A1 — Sign in

Purpose. A1 is the entry point of the Browser Client. A user must be logged in before any offer is shown or any request is sent, and a wrong login shows an error (M8).

Layout. Centered form under the header line: the title "Sign in to LitasCloud", the sub-title "Use your Host or Player account.", the labelled fields "Username" and "Password", a dark primary button "Sign in", the text "New to LitasCloud?" and a secondary button "Create account".

Table 37: A1 — elements and behaviour

Element	Behaviour
Username field	Single-line text. Required. The wireframe shows the sample value "jonas20".
Password field	Single-line text, every character is shown as a dot. Required. The password is sent over TLS (NFR-11).
"Sign in" button	Sends the username and password in the login request of the REST API. On success the server answers with an access token and the account role (Flow 1). For a Player account the client opens P1. As the note under the wireframe states, the same screen leads a Host account to the Host Agent setup; the Host Agent has its own login (Section 3.1).
Error area	Hidden by default. After a failed login it appears between the Password field and the "Sign in" button with the text "Wrong username or password. Try again."
"Create account" button	Opens A2.

8.1.2 A2 — Create account

Purpose. A2 lets a new user register. Every account has a unique username, a password stored as a salted bcrypt hash and exactly one role that is fixed at registration (M8, Table 20).

Layout. Centered form: the title "Create your account", the fields "Username" (placeholder "Choose a unique username") and "Password" (placeholder "Enter a password"), the label "Account type" with a two-option switch "Player" and "Host", a dark primary button "Create account", and the line "Already registered? Sign in".

Table 38: A2 — elements and behaviour

Element	Behaviour
Username field	Required. 3–32 letters, digits or underscores, unique among all accounts (Table 20). The placeholder disappears when the user types.
Password field	Required. Masked like on A1. The server stores only a salted bcrypt hash (NFR-11).
Account type switch	Exactly one of the two options is selected. Only "Player" and "Host" are offered. Operator accounts are managed separately and can never be registered here (Table 19, M8). The role cannot be changed later.
"Create account" button	Sends the registration request (`POST /auth/register`) with username, password and role.
Username error message	Shown directly under the Username field when the username is already in use: "This username is already taken. Choose another."
"Sign in" link	Opens A1.

8.1.3 P1 — Browse available games

Purpose. P1 is the home screen of a signed-in Player and the screen to which every result screen returns. It shows only offers that are AVAILABLE, with the session length, and lets the Player send a join request by pressing "Play" (M2). Before connecting, the Player sees the price, the PC specifications and the recording notice (S4, C5, M13).

Layout. Below the shared header: the title "Available games", the sub-title "Choose a game. Its Host will approve your request.", a grey recording notice, a vertical list of offer cards, and a footer line.

Offer card. A card has a "Play" button on its right side and four lines of text:

- the game name in bold (for example "Minecraft (singleplayer)");
- the line "Host: <username> · <durationMin> minutes from the first video frame", which shows the booked length (M9, `durationMin` of the offering);
- the line "<pricePerMinute> credits/min · <required> credits required and reserved for this session", where *required* is `pricePerMinute` × `durationMin`. This is the amount that is held when the request is made (S4, `heldCredits` of the join request); in the wireframe $2 \times 30 = 60$ and $1 \times 60 = 60$;

- a smaller line with the PC specifications and connection quality: "<cpu> · <gpu> · <ramGb> GB RAM · Upload <uploadMbps> Mbps · Avg latency <latencyMs> ms". The values come from the fields `pcSpecs` and `connection` of the offering (C5, S5, Table 22).

Table 39: P1 — elements and behaviour

Element	Behaviour
Header balance	"<username> | <creditBalance> credits", see Table 36. The balance is only displayed.
Recording notice	Static text, always visible: "Session video is recorded for abuse review. Video only; operator access only. Deleted after 7 days unless a report is open." It informs the Player before connecting, as M13 requires.
Offer list	Loaded from the offering list of the REST API and reloaded every 5 s (`OfferingList.refreshSec`). It contains only offerings with status AVAILABLE (M2). An offer disappears when a Player reserves it, the Host cancels it, or the Host Agent stops its heartbeat (within 10 s, NFR-08).
"Play" button (enabled)	Enabled when the available balance is at least the required credits of the card. Sends `join_request(offeringId, gameId)` over the WebSocket. The server then reserves the Host and holds the credits in one atomic step (NFR-09, Flow 2). When `join_pending` arrives, the client opens P2.
"Play" button (disabled)	Not pressable when the available balance is lower than required. No request is sent, because a Player needs enough credits for the chosen duration before the request is sent (S4).
Credit message	Shown below the list, at the place of the footer line, when the balance is too low: "Not enough credits: this session requires 60 credits before you can request it." In the wireframe the header shows 12 credits.
Footer line	"Only available offers are shown. Credits are charged for time played."

8.1.4 P2 — Waiting for Host approval

Purpose. After "Play", the Host Agent asks the Host for approval. P2 tells the Player that the request is waiting, how long the Host still has to answer and that the credits are only reserved. Nothing is streamed and no input is accepted before approval (M3).

Layout. Centered content under the header: the title "Waiting for the Host to approve…", a spinner, the game name in bold, the line "Host: <username> · Session: <durationMin> minutes", the bold line "Host decision time remaining: mm:ss", a grey box with "60 credits reserved" and "Released if this request is not accepted.", and the button "Cancel request". The header shows "120 available credits", which is the previous balance of 180 minus the 60 held.

Table 40: P2 — elements and behaviour

Element	Behaviour
Spinner	Animated while the request is pending.

Element	Behaviour
Game and Host lines	The game name and the Host username of the chosen offer and the booked length in minutes.
Decision countdown	Counts down from 00:30 once per second (M3). The value 00:22 in the wireframe is a snapshot.
Credits box	Shows the held amount, `pricePerMinute` × `durationMin`, and states that it is released if the request is not accepted (S4).
"Cancel request" button	Sends `cancel_request(requestId)`. The server releases the request, the Host reservation and the credit hold within 2 s and closes the Host's approval dialog (S1). When `request_cancelled` arrives, the client opens P1, where the balance is back to 180 and the Player can send a new request to any Host.
Outcome: approved	`join_approved` opens P3.
Outcome: rejected	`join_rejected` with the reason that the Host declined opens P6 Request declined.
Outcome: no answer	`join_rejected` with the reason that the 30 s timer fired opens P6 Host did not respond.

8.1.5 P3 — Connecting to the game

Purpose. After approval the Host Agent starts the chosen game on a hidden desktop and the two peers set up the connection. P3 shows the deadline for the first video frame and the fact that billing has not started yet (M4, M9). When the connection has to use the TURN relay, P3 says so (S2).

Layout. Centered content: the title, a spinner, the bold line "Starting <game name>", the line "Host: <username>", the line "First-frame deadline: mm:ss remaining" and a grey box with the text "Your booked time and credits start only when the first video frame arrives." The header shows "120 available credits". P3 has no button.

Table 41: P3 — elements and behaviour

Element	Behaviour
Title	"Connecting…" for a direct path. When the connection uses the TURN relay the title is replaced by "Connecting via relay"; everything else on the screen stays the same (S2).
Spinner	Animated until the first video frame is shown or the setup fails.
First-frame deadline	Counts down from 00:10 once per second, measured from the Host's approval (M4, G2). The value 00:06 is a snapshot.
Billing notice	Static text. It tells the Player that the booked time and the charge start at the first frame (M9, S4).
Success	When the first video frame is displayed, the client opens P4 (Section 3.3).
Failure	`session_ended` with the reason `setup_failed` opens P6 The game could not connect. The Player is not charged (Section 3.5).

8.1.6 P4 and P4a — Play the streamed game

Purpose. P4 is the screen on which the Player plays. The browser shows the game video of the Host's PC and sends keyboard and mouse input to it (M4, M5). P4a is the same screen during the last minute of the booking (M9).

Layout. Under the shared header, the video fills a frame with the aspect ratio 16:9. Three controls are laid over the top edge of the video: the button "Leave" on the left, the bold box "Time left: mm:ss" in the middle, and the box "Latency: <n> ms | FPS: <n>" on the right. A white strip along the bottom edge shows the text "Game video is recorded for abuse review. Operators only; no audio." The header shows "120 available credits".

Table 42: P4 and P4a — elements and behaviour

Element	Behaviour
Video	Plays the video track of the Host Agent (720p, 30 FPS target, NFR-01) in a plain video element without any plugin (NFR-14). The video contains only the game, never the Host's desktop (M11). There is no sound.
Time left	Remaining booked time as mm:ss. It starts at the booked length at the first video frame and counts down once per second (M9). The values 18:00, 17:45 and 01:00 in the wireframes are snapshots.
Latency and FPS	Round-trip latency in milliseconds, measured by a ping sent every second over the data channel `control` and answered by the Host Agent, and the number of video frames displayed per second. Both are updated every second (M7). The targets are a latency below 100 ms and 30 FPS (NFR-01, NFR-02).
Recording strip	Static notice for the whole session. It tells the Player that the video is recorded and that there is no audio (M13).
Keyboard input	While P4 is open, key presses and releases of letters, digits, arrow keys, Space, Enter, Escape and Shift are sent over the data channel. All other keys, and every combination that uses Ctrl, Alt or the Windows key, are not sent (M5).
Mouse input	Mouse movement, left and right clicks and scroll are sent over the data channel. The Host Agent applies the input only while the game window is the target (M5, M11).
"Leave" button	Always visible. Sends `session_end(player_left)` and opens P6 You left the session. The Host Agent stops capture, releases all held keys and closes the game (M6). No confirmation dialog is shown, because leaving never blocks the Host (S3).
Host ends the session	`session_ended` with the reason `host_disconnected` opens P6 The Host ended the session (M6).

State P4a (one minute left). When the server sends `time_warning` with 60 s left, a banner appears below the top controls inside the video: "1 minute left. The session and game will close automatically." Time left shows 01:00 at the moment the banner appears (M9). There is no button to extend the session in this version (C6 is not implemented). When the time reaches zero the server ends the session with the reason `duration_elapsed` and the client opens P6 Session complete; this end carries no penalty and the Player pays the full booked duration (Table 9).

8.1.7 P5 — Recovering a dropped connection

Purpose. If the heartbeat stops, P5 tells the Player that the connection is being recovered, that the controls are paused and that the time keeps running (S6).

Trigger and layout. The Browser Client sends a ping every second and the Host Agent answers with a pong. When no answer has arrived for 5 s, a white card appears in the centre of the video: the title "Reconnecting…", the line "mm:ss left to recover the connection", the line "Controls are paused and all keys released." and the bold line "Your booked time keeps counting." The top controls, the video frame and the recording strip stay visible behind the card. The header shows "120 available credits".

Table 43: P5 — elements and behaviour

Element	Behaviour
Recovery countdown	Counts down from 00:20 once per second from the moment the card appears; 00:15 in the wireframe is a snapshot. The recovery window is 20 s (S6, NFR-07).
Paused controls	The client stops sending keyboard and mouse events. The Host Agent blocks input and releases all held keys on its side (S6, M5).
Time left	Keeps counting down, because the booked time and the billing continue during the drop (S6).
"Leave" button	Stays available and works as on P4: it ends the session with the reason `player_left` and opens P6 You left the session.
Recovered	If the heartbeat is answered again within the window, the card disappears and the stream resumes in state P4 (or P4a, if the last minute has begun).
Not recovered	If the heartbeat does not return within 20 s, the session ends with the reason `connection_lost` and the client opens P6 Connection lost (Table 9).

8.1.8 P6 — Result screens

Purpose. P6 is the last screen of every visit. One layout with seven variants tells the Player why the visit ended, whether credits were charged and what happens next. The variants are grouped by the moment at which the visit ended: before any video was shown (the first three rows of Table 44) and after the stream started (the last four rows).

Common layout. Under the shared header: a bold title, one line of text under it, a grey box with the explanation of the credits, and a dark button "Back to games". The variant Session complete replaces the grey box by a summary card and adds one line below it.

Table 44: P6 variants: texts and triggers

Title	**Line under the title**	**Grey box**	**Trigger and header**
Request declined	The Host declined your request.	No credits charged. Your 60-credit reservation has been released.	The Host rejected the request (P2). Header: 180 credits.
Host did not respond	The Host did not answer within 30 seconds.	No credits charged. Your reservation has been released. Try another available game.	The 30 s timer fired (P2). Header: 180 credits.
The game could not connect	No video arrived in time.	No credits charged. Your reservation has been released. Please choose an available game and try again.	End reason `setup_failed` (P3). Header: 180 credits.
Session complete	Your 30-minute booking has ended.	Summary card (below) and the line "The game is closed and the credit reservation is settled."	End reason `duration_elapsed` (P4a). Header: 120 credits.
You left the session	Your connection to the game is closed.	Credits are settled for time played. Unused reserved credits are released.	"Leave" pressed (P4, P4a, P5). Header: username only.
The Host ended the session	This booking ended early.	You are charged only for time played. Unused reserved credits are released.	End reason `host_disconnected` (P4, P4a). Header: username only.
Connection lost	The connection could not be restored.	The session has ended. You are charged only for time played; unused reserved credits are released.	End reason `connection_lost` (P5). Header: username only.

Variable parts. The number in "60-credit reservation" is the amount that was held for the request. The number in "30-minute booking" is the booked length of the offering.

Summary card of Session complete. A grey card with five rows, label on the left and value on the right: "Game" ("Minecraft (singleplayer)"), "Host" ("tadas_pc"), "Time played" ("30:00"), "Credits charged" in bold ("60 credits") and "Your new balance" in bold ("120 credits"). The game name and the Host username are those of the finished session. The note under the wireframe gives the example: a complete 30-minute session at 2 credits per minute costs 60 credits, and no real-money action exists (W3).

How the charge is computed. Credits are counted from the first video frame, as described in Section 3.5 and S4. In the example the session lasted 30:00, so the Player is charged $30 \times 2 = 60$ credits. For an early end the Player pays only for the time played and the unused part of the hold is released; when no video was ever shown (the variants Request declined, Host did not respond and The game could not connect) nothing is charged and the whole hold is released (Table 9).

Table 45: P6 — elements and behaviour

Element	Behaviour
Header balance	Follows Table 36. In the three variants that end after play (You left the session, The Host ended the session, Connection lost) the header shows only the username.
Title and text lines	Fixed texts of Table 44. Every text says what happened and what the Player can do next (NFR-15).
“Back to games” button	Opens P1. The button never leads to a code-entry screen, because there are no pairing codes (M12).
Session state	When P6 is shown after play, the session is ENDED and the Host has been told to stop input, release all keys, stop capture and close the game (M6, Section 3.4). The Player cannot return to the stream.

Requirement coverage. Table 46 shows which Player screens implement each requirement of Section 2.5.

Table 46: Requirements covered by the Player screens

ID	Requirement	Screens
M2	Browse available games and press play	P1, P2
M3	Host approval, rejection and timeout shown to the Player	P2, P6
M4	See the game in the browser, first frame within 10 s	P3, P4, P6
M5	Keyboard and mouse control with a whitelist	P4
M6	Session end, Player’s “Leave” button	P4, P5, P6
M7	Latency and FPS on screen	P4, P4a, P5
M8	Register and log in	A1, A2
M9	Booked time, warning one minute before the end	P1, P3, P4, P4a, P6
M13	Player told about the recording before connecting and during play	P1, P4
S1	Cancel a waiting request	P2
S2	“Connecting via relay”	P3
S4	Credits: price, hold, charge for time played	P1, P2, P6, header
S6	Recover a short connection drop	P5
C5	PC specifications before pressing play	P1
NFR-14, NFR-15	Browser only; messages say what to do next	all screens

Not drawn in this version. The following optional features of Section 2.5 have no Player wireframe because they are not part of the first version: game sound (C1), gamepad (C2), video quality presets (C3), fullscreen (C4), session extension (C6), ratings and reports (C7) and session history (C9). A game filter and a credit top-up screen are not drawn either.

8.2 Operator Wireframes

[Link to Operator Wireframe in Figma]

This subsection documents the two screens of the Operator console: **O1** Review and lift Host blocks and **O2** Review session recordings. Both are drawn inside a browser window whose address bar reads "LitasCloud — Operator console". The note under O1 names requirement S3 and the note under O2 names requirement M13 (Table 47).

Table 47: Overview of the Operator screens

ID	Screen	Req.	Note under the wireframe
O1	Review and lift Host blocks	S3	Operator-only review. A Player leaving or a connection failure never triggers the Host penalty.
O2	Review session recordings	M13	Recordings are not shown to Hosts or Players. Recording failures are flagged without ending gameplay.

Shared header and tabs. Both screens have the same header. On the left is the title "LitasCloud / Operators". On the right are the username of the signed-in Operator ("operator01" in the wireframes) and the button "Sign out", with a line below. Under the header are two tab buttons, "Host blocks" and "Recordings". The tab of the open screen is drawn dark: "Host blocks" on O1 and "Recordings" on O2. The other tab is drawn light and leads to the other screen.

8.2.1 O1 — Review and lift Host blocks

Purpose. O1 lets an Operator review the reason for a Host's restriction before lifting it (S3). As the note under the wireframe states, the review is Operator-only, and a Player leaving or a connection failure never triggers the Host penalty.

Layout. Under the tabs: the title "Host blocks", the sub-title "Review the reason before lifting a restriction.", a table, and below it a bordered detail panel.

Table 48: O1 — elements

Element	Content in the wireframe
Table columns	"HOST", "REASON" and "TIME REMAINING".
Row 1 (selected)	"tadas_pc", "Session ended before booked time", "00:59:42". The row has a grey background and the Host name is bold.
Row 2	"egis_pc", "Repeat early ending within 30 days", "21:14:09".
Detail panel, title	Bold line "Selected Host: tadas_pc", the Host of the selected row.
Detail panel, reason	"Reason: early Disconnect with 18 minutes remaining."
Detail panel, length	"Block length: 1 hour"
"Lift block" button	Dark button on the right side of the detail panel.

8.2.2 O2 — Review session recordings

Purpose. O2 lets an Operator review recordings of sessions (M13). As the note under the wireframe states, recordings are not shown to Hosts or Players, and a recording failure is flagged without ending

gameplay.

Layout. Under the tabs: the title “Session recordings”, the sub-title “Operator access only · Video only · Automatic 7-day retention”, a table, and below it two areas side by side: a video frame on the left and the details of the selected session on the right.

Table 49: O2 — elements

Element	Content in the wireframe
Table columns	“SESSION”, “HOST / PLAYER”, “RECORDING” and “DELETE ON”.
Row 1 (selected)	“7a8d…91c4”, “tadas_pc / jonas20”, “Ready”, “13 Nov 2026”. The row has a grey background.
Row 2	“b046…d129”, “egis_pc / player02”, “Failed — flagged”, “No file”.
Video frame	A grey frame with the text “RECORDED GAME VIDEO”. A control bar along its bottom edge shows a play button and the position “00:00 / 30:00”.
Detail area, title	Bold “Selected session”.
Detail area, identifier	The full session identifier on two lines: “7a8d05c8-4610-4d37-” and “8c15-e1a78b2f91c4”.
Detail area, users	“Host: tadas_pc · Player: jonas20”
Detail area, times	“Started: 6 Nov 2026, 18:00”, “Ended: 6 Nov 2026, 18:30” and “Delete after: 13 Nov 2026, 18:30”.

8.3 Host Agent Wireframes

[Link to Host Agent Wireframe in Figma]

This subsection documents the screens of the Host Agent as they are drawn in the wireframes. The identifiers are H1 to H9; H3a is the failed state of H3, and H6 is drawn in three states.

Conventions in the wireframes. Every screen is drawn as a desktop window with a title bar that reads “LitasCloud Host Agent” and shows a dash symbol at its right end. Inside the window, every screen has the same header: the name “LitasCloud” on the left, the Host’s username and credit balance on the right (“tadas_pc | 120 credits”), and a line below. Only H9 shows another balance, “tadas_pc | 177 credits”. Under each wireframe a one-line note lists the requirements it implements.

Screen overview. Table 50 lists the screens with the note printed under each wireframe.

Table 50: Overview of the Host Agent screens

ID	Screen	Note under the wireframe
H1	Set up an offering	M1, M9, M10, M13, S4, S5. Starts a self-test, then publishes one offering when all checks pass.
H2	Manage allowed games	M10. Only the Host sees executable paths. Example paths are placeholders, not tested launch commands.

ID	Screen	Note under the wireframe
H3	PC readiness check	S5. Success publishes the offering automatically. A failed check offers Retry and Back to setup.
H3a	PC check failed	S5. Failed checks explain the problem and prevent an offering from becoming Available.
H4	Offering is available	M1, M3, M9. Rejection, timeout or a cancelled request returns to this Available offering.
H5	Approve a Player request	M3. The request expires after 30 seconds; Player Cancel also closes this dialog.
H6	Active hosting session (three states)	M6, M9, M11, M13. No game or desktop preview. Early Disconnect opens the S3 penalty confirmation.
H7	Confirm an early disconnect	S3. Confirmation precedes the penalty. Player leaving or a connection failure never blocks the Host.
H8	Hosting is temporarily blocked	S3. Show the reason and remaining block duration. A repeat early ending uses a 24-hour block.
H9	Session result	M6, S4. Completed 30-minute example: 60 credits charged = 57 Host earnings + 3 service fee.

8.3.1 H1 — Set up an offering

Layout. Under the header: the title “Offer your PC”, the sub-title “Choose the games and the length of each session.”, a list of games with a button on its right, a session duration area with a price area on its right, a grey notice, and at the bottom a button with a hint beside it.

Table 51: H1 — elements

Element	Content in the wireframe
Game list	Bold label “Games available to Players”. Two check boxes, both checked: “Minecraft (singleplayer)” and “Platformer Demo”.
“Manage games” button	Light button to the right of the game list.
Session duration	Bold label “Session duration”. A drop-down field that shows “30 minutes” and a small down arrow. Under it the small text “Allowed: 15 minutes to 4 hours.”
Price area	Bold line “Price: 2 credits / minute”, the line “Full-session reward: 57 credits” and the small text “After the 5% service fee.”
Recording notice	Grey box: “Session video is recorded for abuse review. Video only; operator access only. Deleted after 7 days unless a report is open.”
“Start hosting” button	Dark button at the bottom left. Beside it the hint “PC checks run before your offer goes live.”

8.3.2 H2 — Manage allowed games

Layout. Under the header: the title “Your allowed games”, the sub-title “Only games in this list can be launched for a Player.”, a table, a row of two input fields, a button, and a button at the bottom right.

Table 52: H2 — elements

Element	Content in the wireframe
Table columns	“GAME” and “PROGRAM PATH — HOST ONLY”.
Row 1	“Minecraft (singleplayer)”, `C:\Games\Minecraft\game.exe`.
Row 2	“Platformer Demo”, `C:\Games\Platformer\game.exe`.
Field “Game name”	Text field with the placeholder “New game name”.
Field “Program path”	Text field with the placeholder `C:\Games\...\game.exe`.
“Add game” button	Light button below the two fields.
“Done” button	Dark button at the bottom right.

8.3.3 H3 — PC readiness check

Layout. Under the header: the title “Checking your PC…”, the sub-title “Your offer stays private until all checks pass.”, three check cards, and two lines of text.

Table 53: H3 — elements

Element	Content in the wireframe
Check card 1	“Server connection”, result “Passed”.
Check card 2	“Upload speed: 8.7 Mbps”, result “Passed”.
Check card 3	“Game capture”, result “Checking…”
Text below the cards	“Minimum upload speed: 4 Mbps.” and “Once ready, your games appear as Available.”

8.3.4 H3a — PC check failed

Layout. Under the header: the title “Your PC is not ready yet”, a grey box with a dark border, three check cards, a line of text, and two buttons.

Table 54: H3a — elements

Element	Content in the wireframe
Message box	“Your offering was not created. Fix the failed check and try again.”
Check card 1	“Server connection”, result “Passed”.
Check card 2	“Upload speed: 2.1 Mbps”, result “Failed”.
Check card 3	“Game capture”, result “Passed”.
Text below the cards	“Upload needs at least 4 Mbps. Pause other uploads or check your connection.”
“Retry checks” button	Dark button.

Element	Content in the wireframe
“Back to setup” button	Light button beside “Retry checks”.

8.3.5 H4 — Offering is available

Layout. Under the header: a badge, the title “Waiting for a Player request”, the sub-title “Your selected games are listed in the Player catalog.”, a grey box, a line of text, and a button.

Table 55: H4 — elements

Element	Content in the wireframe
Badge	“AVAILABLE”.
Grey box	Bold line “Minecraft (singleplayer) · Platformer Demo”, the line “Session length: 30 minutes” and the line “2 credits / minute · Full-session reward: 57 credits”.
Text below the box	“The booked timer starts when the Player sees the first frame.”
“Cancel offering” button	Light button.

8.3.6 H5 — Approve a Player request

Layout. Under the header: the title “A Player wants to connect” with a rounded box on its right, a grey box with four rows, two lines of text, and two buttons.

Table 56: H5 — elements

Element	Content in the wireframe
Rounded box	“00:22 left”.
Row “Player”	“jonas20” (bold).
Row “Game”	“Minecraft (singleplayer)” (bold).
Row “Session length”	“30 minutes”.
Row “Price”	“2 credits / minute”.
Text below the box	“Only approval starts the game and the stream.” and, in smaller grey text, “Rejecting or not answering keeps your offer Available.”
“Approve” button	Dark button.
“Reject” button	Light button beside “Approve”.

8.3.7 H6 — Active hosting session

Layout. Under the header: a badge, the title “Player connected”, the line “jonas20 · Minecraft (singleplayer)”, a grey box with two values, four lines of text, and a dark button. As the note under the wireframe states, there is no game or desktop preview. H6 is drawn in three states that differ in the badge, the time value and the last line of text (Table 58).

Table 57: H6 — elements common to all states

Element	Content in the wireframe
Grey box, left value	Label “TIME REMAINING” with a large value in `mm:ss`.
Grey box, right value	Label “BOOKED SESSION” with the value “30 minutes”.
Text line 1	Bold: “The game runs separately from your desktop.”
Text line 2	Grey: “You can keep working. Only the game is streamed.”
Text line 3	“Session video is being recorded for abuse review.”
Last line	Differs by state, see Table 58.
“Disconnect…” button	Dark button at the bottom left. As the note under the wireframe states, an early Disconnect opens the S3 penalty confirmation (H7).

Table 58: H6 — the three states

State	Badge	Time remaining	Last line of text
Active	“SESSION ACTIVE”	18:00	Small grey text: “A warning appears when 1 minute remains.”
Player reconnecting	“RECON-NECTING”	17:45	Grey box: “Player reconnecting — input paused. Booked time keeps counting.”
One minute left	“SESSION ACTIVE”	01:00	Grey box: “1 minute left. The session and game will close automatically.”

8.3.8 H7 — Confirm an early disconnect

Layout. Under the header: a large grey panel that contains the title “End the session early?”, four lines of text and two buttons.

Table 59: H7 — elements

Element	Content in the wireframe
Line 1	“There are 18 minutes left in this booking.”
Line 2	Bold: “Disconnecting now blocks new hosting for 1 hour.”
Line 3	“A repeat early ending within 30 days blocks hosting for 24 hours.”
Line 4	“The Player pays only for time played. Your stream and input stop within 2 seconds after you confirm.”
“Keep hosting” button	Light button.
“Disconnect now” button	Dark button beside “Keep hosting”.

8.3.9 H8 — Hosting is temporarily blocked

Layout. Under the header: a badge, the line “You can host again in”, a large time value, a grey box with a dark border, two grey lines of text, and a disabled button.

Table 60: H8 — elements

Element	Content in the wireframe
Badge	“HOSTING BLOCKED”.
Time value	“00:59:42”, in the format `hh:mm:ss`, in large type under “You can host again in”.
Grey box	“Your last session ended before the booked time. New hosting is blocked for 1 hour.”
Text lines	Grey: “An operator can review and lift this block.” and “Your account balance remains available.”
“Start hosting” button	Drawn grey (disabled).

8.3.10 H9 — Session result

Layout. Under the header, which shows “tadas_pc | 177 credits”: the title “Session complete”, a summary line, a grey card with four rows, a line of text, and a dark button.

Table 61: H9 — elements

Element	Content in the wireframe
Summary line	“jonas20 · Minecraft (singleplayer) · 30:00 played”.
Card row 1	“Player credits spent”, “60 credits”.
Card row 2	“Service fee (5%)”, “3 credits”.
Card row 3 (bold)	“Your credits earned”, “57 credits”.
Card row 4 (bold)	“New balance”, “177 credits”.
Text below the card	“The game is closed. You can create another offering.”
“Create new offering” button	Dark button.

As the note under the wireframe states, the example is a completed 30-minute session in which 60 credits are charged, which is 57 credits of Host earnings plus a service fee of 3 credits.